

CIS 6270

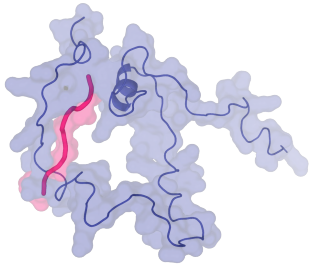
Module 4: Discrete Diffusion

Lecture 4.5: Discrete Flow Matching with Jump Rates

Tue, Oct 20, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



Should every token disappear at the same rate?

MD4 and GenMD4

Simplified and Generalized Masked Diffusion for Discrete Data

Jiaxin Shi*, **Kehang Han***, **Zhe Wang**, **Arnaud Doucet**, **Michalis K. Titsias**
Google DeepMind

Shi,..,Titsias. *NeurIPS* (2024)

Suppose each token has its own masking rate, as in GenMD4

Suppose each token has its own masking rate, as in GenMD4

In every model so far, we mask each token at the same rate $\beta(t)$.

Suppose each token has its own masking rate, as in GenMD4

In every model so far, we mask each token at the same rate β(t).

In GenMD4, we give each token its own rate, so rare tokens can survive longer:

$$R_t(a, m) = \beta_a(t)$$

Suppose each token has its own masking rate, as in GenMD4

In every model so far, we mask each token at the same rate β(t).

In GenMD4, we give each token its own rate, so rare tokens can survive longer:

$$R_t(a, m) = \beta_a(t)$$

The rate $R_t(a, m)$ at which the clean token a jumps to the mask token m equals its own schedule $\beta_a(t)$.

Suppose each token has its own masking rate, as in GenMD4

In every model so far, we mask each token at the same rate β(t).

In GenMD4, we give each token its own rate, so rare tokens can survive longer:

$$R_t(a, m) = \beta_a(t)$$

The rate $R_t(a, m)$ at which the clean token a jumps to the mask token m equals its own schedule $\beta_a(t)$.

The forward marginal has a closed form as before, with a token-dependent $\alpha_t(a)$, and facts 1 to 3 still hold.

Suppose each token has its own masking rate, as in GenMD4

In every model so far, we mask each token at the same rate β(t).

In GenMD4, we give each token its own rate, so rare tokens can survive longer:

$$R_t(a, m) = \beta_a(t)$$

The rate $R_t(a, m)$ at which the clean token a jumps to the mask token m equals its own schedule $\beta_a(t)$.

The forward marginal has a closed form as before, with a token-dependent $\alpha_t(a)$, and facts 1 to 3 still hold.

The generalized loss has a state-dependent weight, and we learn the schedule itself with the same bound.

Suppose each token has its own masking rate, as in GenMD4

In every model so far, we mask each token at the same rate β(t).

In GenMD4, we give each token its own rate, so rare tokens can survive longer:

$$R_t(a, m) = \beta_a(t)$$

The rate $R_t(a, m)$ at which the clean token a jumps to the mask token m equals its own schedule $\beta_a(t)$.

The forward marginal has a closed form as before, with a token-dependent $\alpha_t(a)$, and facts 1 to 3 still hold.

A slower masking rate keeps a token visible longer, so other positions can use it as context.

MD4 and GenMD4: which bases stay visible?

Consider one time unit of a toy corruption segment for visible A and G. Compare a shared constant masking rate of 1 with token-specific rates of 2 for A and 0.5 for G.

MD4 and GenMD4: which bases stay visible?

Consider one time unit of a toy corruption segment for visible A and G. Compare a shared constant masking rate of 1 with token-specific rates of 2 for A and 0.5 for G.

$$\text{shared rate:} \qquad \alpha_1(\mathrm{A}) = \alpha_1(\mathrm{G}) = e^{-1} \approx 0.368$$

MD4 and GenMD4: which bases stay visible?

Consider one time unit of a toy corruption segment for visible A and G. Compare a shared constant masking rate of 1 with token-specific rates of 2 for A and 0.5 for G.

$$\text{shared rate:} \qquad \alpha_1(\mathrm{A}) = \alpha_1(\mathrm{G}) = e^{-1} \approx 0.368$$

With a shared rate, each base has a 36.8% chance of still being visible.

MD4 and GenMD4: which bases stay visible?

Consider one time unit of a toy corruption segment for visible A and G. Compare a shared constant masking rate of 1 with token-specific rates of 2 for A and 0.5 for G.

$$\text{shared rate:} \qquad \alpha_1(\mathrm{A}) = \alpha_1(\mathrm{G}) = e^{-1} \approx 0.368$$

With a shared rate, each base has a 36.8% chance of still being visible.

$$\text{token-specific rates:} \qquad \alpha_1(\mathrm{A}) = e^{-2} \approx 0.135, \quad \alpha_1(\mathrm{G}) = e^{-0.5} \approx 0.607$$

MD4 and GenMD4: which bases stay visible?

Consider one time unit of a toy corruption segment for visible A and G. Compare a shared constant masking rate of 1 with token-specific rates of 2 for A and 0.5 for G.

$$\text{shared rate:} \qquad \alpha_1(\mathrm{A}) = \alpha_1(\mathrm{G}) = e^{-1} \approx 0.368$$

With a shared rate, each base has a 36.8% chance of still being visible.

$$\text{token-specific rates:} \qquad \alpha_1(\mathrm{A}) = e^{-2} \approx 0.135, \quad \alpha_1(\mathrm{G}) = e^{-0.5} \approx 0.607$$

Now A survives 13.5% of the time and G survives 60.7% of the time.

MD4 and GenMD4: which bases stay visible?

Consider one time unit of a toy corruption segment for visible A and G. Compare a shared constant masking rate of 1 with token-specific rates of 2 for A and 0.5 for G.

$$\text{shared rate:} \qquad \alpha_1(\mathrm{A}) = \alpha_1(\mathrm{G}) = e^{-1} \approx 0.368$$

With a shared rate, each base has a 36.8% chance of still being visible.

$$\text{token-specific rates:} \qquad \alpha_1(\mathrm{A}) = e^{-2} \approx 0.135, \quad \alpha_1(\mathrm{G}) = e^{-0.5} \approx 0.607$$

Now A survives 13.5% of the time and G survives 60.7% of the time.

A token-dependent schedule controls which information remains available during corruption.

It has a continuous-time bound with a weight that depends on the token

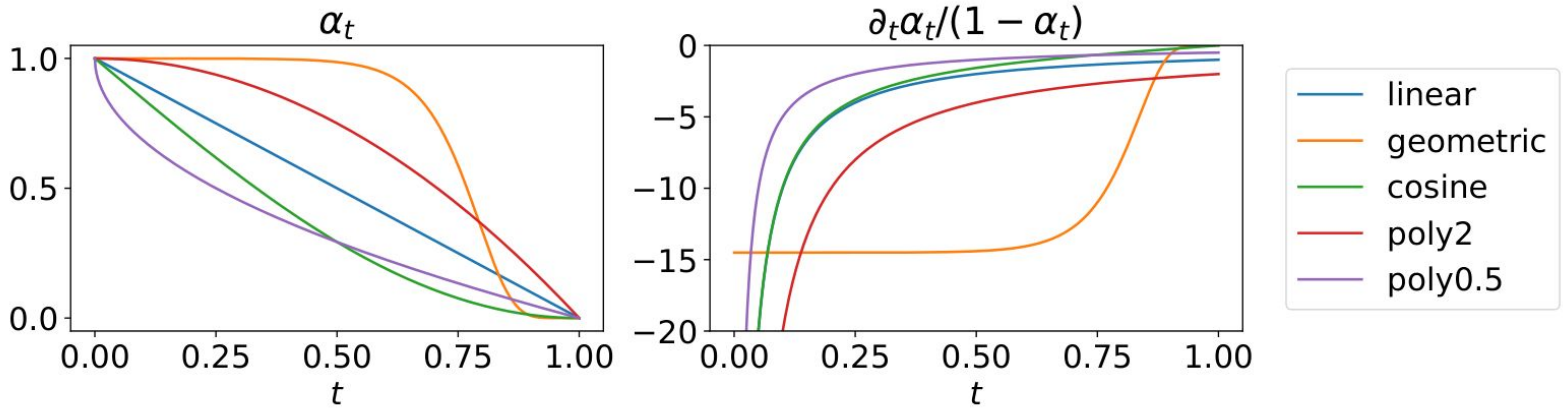


Figure 1: Masking schedules in the literature: (Left) α_t; (Right) weight of the cross-entropy loss w.r.t. t; Equations for these schedules are given in Tab. 4 in Appendix.

In the generalized model, GenMD4, we learn the per-token schedule.

Shi,..,Titsias. *NeurIPS* (2024)

Here is the Text8 result in Table 2 of the paper

Table 2: Bits Per Character (BPC) on Text8 test set. All models use standard 12-layer transformers similar to GPT-2 small [57] except Discrete Flow which uses 8×3 layers.

Method	BPC (↓)
Continuous Diffusion	
Plaid [22] (Our impl.)	≤ 1.48
BFN [26]	≤ 1.41
Any-order Autoregressive	
ARDM [48]	≤ 1.43
MAC [61]	≤ 1.40
Autoregressive	
IAF/SCF [62]	1.88
AR Argmax Flow [15]	1.39
Discrete Flow [59]	**1.23**
Transformer AR [14]	**1.23**
Discrete Diffusion	
Mult. Diffusion [15]	≤ 1.72
D3PM Uniform [14]	≤ 1.61
D3PM Absorb [14]	≤ 1.45
SEDD Absorb [32]	≤ 1.39
MD4 (Ours)	$\leq$ **1.37**
GenMD4 (Ours)	$\leq$ **1.34**

Shi,..,Titsias. *NeurIPS* (2024)

Absorbing D3PM has a Text8 bound of 1.45 bits per character

Table 2: Bits Per Character (BPC) on Text8 test set. All models use standard 12-layer transformers similar to GPT-2 small [57] except Discrete Flow which uses 8×3 layers.

Method	BPC (↓)
Continuous Diffusion	
Plaid [22] (Our impl.)	≤ 1.48
BFN [26]	≤ 1.41
Any-order Autoregressive	
ARDM [48]	≤ 1.43
MAC [61]	≤ 1.40
Autoregressive	
IAF/SCF [62]	1.88
AR Argmax Flow [15]	1.39
Discrete Flow [59]	**1.23**
Transformer AR [14]	**1.23**
Discrete Diffusion	
Mult. Diffusion [15]	≤ 1.72
D3PM Uniform [14]	≤ 1.61
D3PM Absorb [14]	≤ 1.45
SEDD Absorb [32]	≤ 1.39
MD4 (Ours)	$\leq$ **1.37**
GenMD4 (Ours)	$\leq$ **1.34**

Shi,..,Titsias. *NeurIPS* (2024)

MD4 reaches 1.37 with the same transformer

Table 2: Bits Per Character (BPC) on Text8 test set. All models use standard 12-layer transformers similar to GPT-2 small [57] except Discrete Flow which uses 8×3 layers.

Method	BPC (↓)
Continuous Diffusion	
Plaid [22] (Our impl.)	≤ 1.48
BFN [26]	≤ 1.41
Any-order Autoregressive	
ARDM [48]	≤ 1.43
MAC [61]	≤ 1.40
Autoregressive	
IAF/SCF [62]	1.88
AR Argmax Flow [15]	1.39
Discrete Flow [59]	**1.23**
Transformer AR [14]	**1.23**
Discrete Diffusion	
Mult. Diffusion [15]	≤ 1.72
D3PM Uniform [14]	≤ 1.61
D3PM Absorb [14]	≤ 1.45
SEDD Absorb [32]	≤ 1.39
MD4 (Ours)	$\leq$ **1.37**
GenMD4 (Ours)	$\leq$ **1.34**

Shi,..,Titsias. *NeurIPS* (2024)

GenMD4, with learned per-token rates, reaches 1.34

Table 2: Bits Per Character (BPC) on Text8 test set. All models use standard 12-layer transformers similar to GPT-2 small [57] except Discrete Flow which uses 8×3 layers.

Method	BPC (↓)
Continuous Diffusion	
Plaid [22] (Our impl.)	≤ 1.48
BFN [26]	≤ 1.41
Any-order Autoregressive	
ARDM [48]	≤ 1.43
MAC [61]	≤ 1.40
Autoregressive	
IAF/SCF [62]	1.88
AR Argmax Flow [15]	1.39
Discrete Flow [59]	**1.23**
Transformer AR [14]	**1.23**
Discrete Diffusion	
Mult. Diffusion [15]	≤ 1.72
D3PM Uniform [14]	≤ 1.61
D3PM Absorb [14]	≤ 1.45
SEDD Absorb [32]	≤ 1.39
MD4 (Ours)	$\leq$ **1.37**
GenMD4 (Ours)	$\leq$ **1.34**

Shi,..,Titsias. *NeurIPS* (2024)

Choosing the rates well is worth as much as changing the model

Table 2: Bits Per Character (BPC) on Text8 test set. All models use standard 12-layer transformers similar to GPT-2 small [57] except Discrete Flow which uses 8×3 layers.

Method	BPC (↓)
Continuous Diffusion	
Plaid [22] (Our impl.)	≤ 1.48
BFN [26]	≤ 1.41
Any-order Autoregressive	
ARDM [48]	≤ 1.43
MAC [61]	≤ 1.40
Autoregressive	
IAF/SCF [62]	1.88
AR Argmax Flow [15]	1.39
Discrete Flow [59]	**1.23**
Transformer AR [14]	**1.23**
Discrete Diffusion	
Mult. Diffusion [15]	≤ 1.72
D3PM Uniform [14]	≤ 1.61
D3PM Absorb [14]	≤ 1.45
SEDD Absorb [32]	≤ 1.39
MD4 (Ours)	$\leq$ **1.37**
GenMD4 (Ours)	$\leq$ **1.34**

Shi,..,Titsias. *NeurIPS* (2024)

Could we keep several tokens plausible until the surrounding sequence makes the choice clearer?

Simplex Diffusion Models

SIMPLEX DIFFUSION MODELS

Justin Deschenaux
Google DeepMind
EPFL

Alexandre Galashov
Google DeepMind
UCL Gatsby

Andrew Campbell
Google DeepMind

Li Kevin Wenliang
Google DeepMind

James Thornton
Google DeepMind

Arnaud Doucet
Google DeepMind

Valentin De Bortoli
Google DeepMind

Deschenaux,..,De Bortoli. *arXiv* (2026)

So far, we have sampled a token at every step

So far, we have sampled a token at every step

In masked and uniform diffusion, we sample a token in each reverse step, and we discard the denoiser’s posterior once that sample is drawn.

So far, we have sampled a token at every step

In masked and uniform diffusion, we sample a token in each reverse step, and we discard the denoiser’s posterior once that sample is drawn.

This is called information collapse, because uncertainty does not carry from one step to the next.

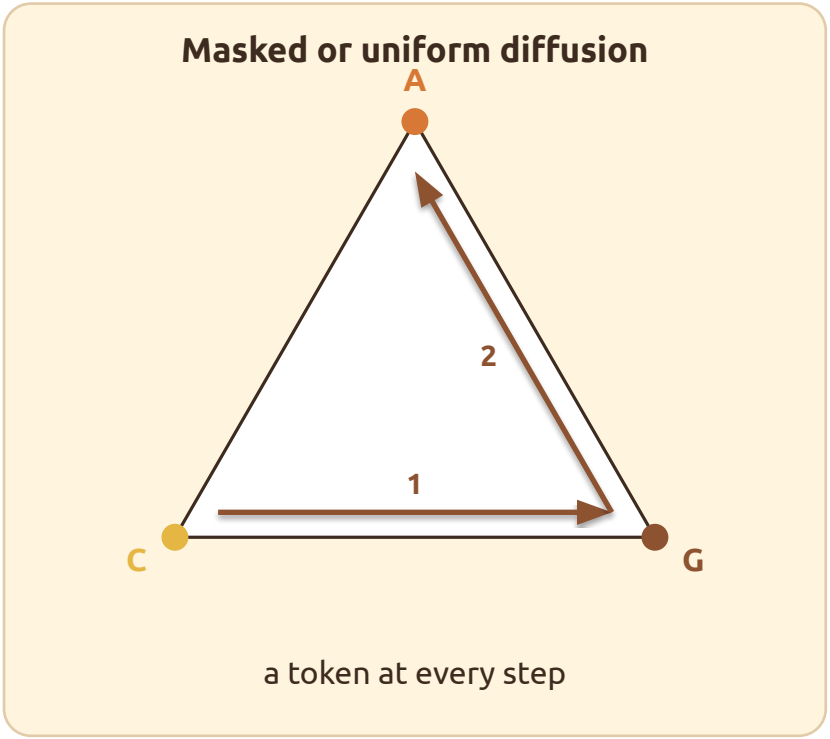
So far, we have sampled a token at every step

In masked and uniform diffusion, we sample a token in each reverse step, and we discard the denoiser’s posterior once that sample is drawn.

This is called information collapse, because uncertainty does not carry from one step to the next.

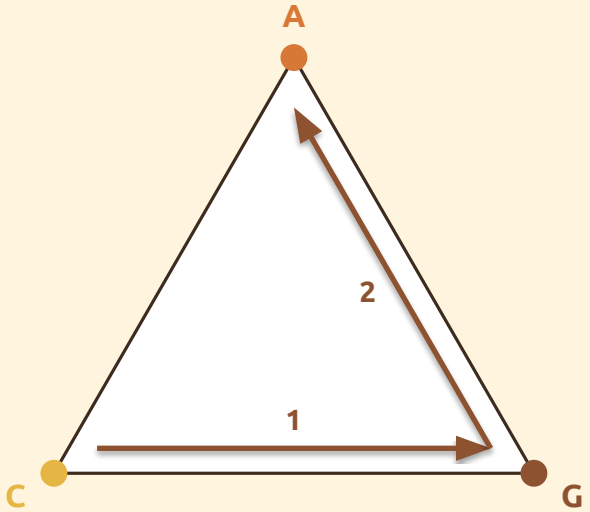
In Simplex Diffusion Models, we keep a probability vector at every position, so the state itself holds the uncertainty.

Masked and uniform diffusion move between the corners of the simplex



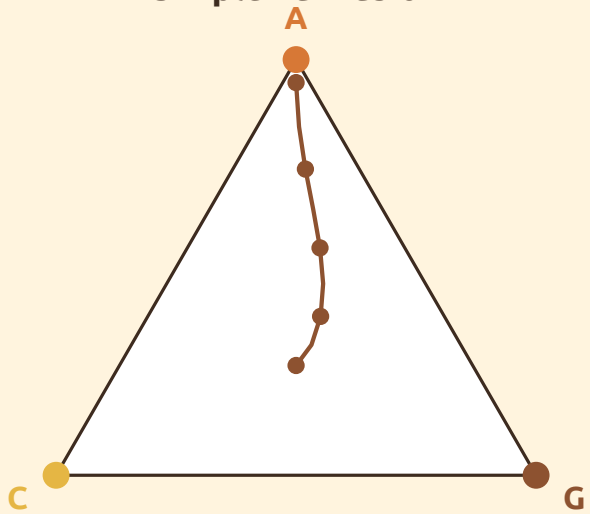
In simplex diffusion, a belief moves through its interior

Masked or uniform diffusion



a token at every step

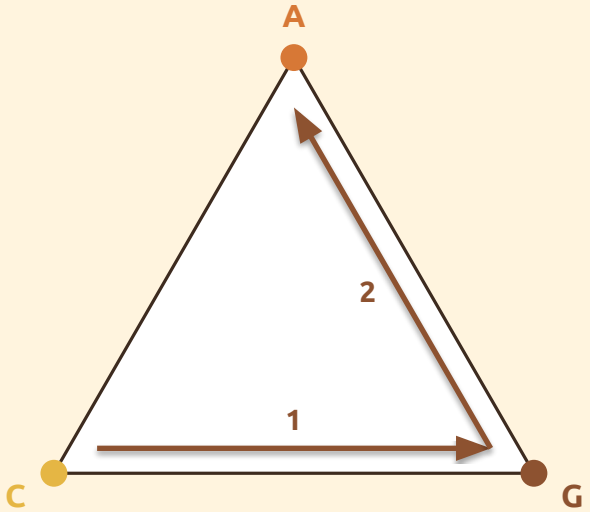
Simplex diffusion



a belief at every step

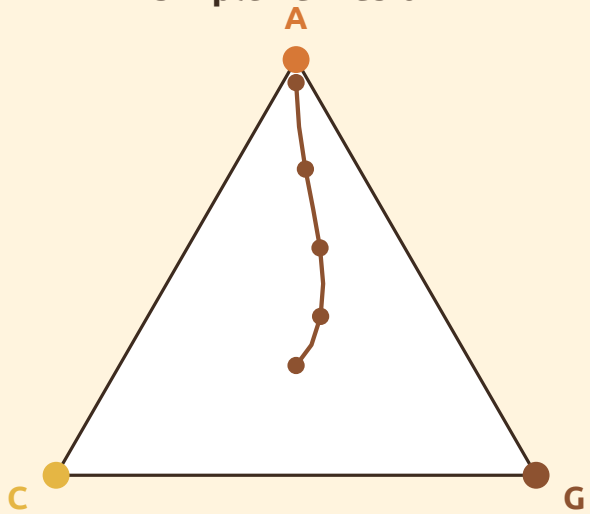
The state space changes from tokens to beliefs

Masked or uniform diffusion



a token at every step

Simplex diffusion



a belief at every step

First, we need a forward process for the beliefs

First, we need a forward process for the beliefs

The noisy belief P_t at a position fluctuates around our forward marginal:

$$\boxed{\mathbb{E}\big[P_t \mid x_0\big] = \alpha_t \, e_{x_0} + (1 - \alpha_t)\, \pi}$$

First, we need a forward process for the beliefs

The noisy belief P_t at a position fluctuates around our forward marginal:

$$\boxed{\mathbb{E}\left[P_t \mid x_0\right] = \alpha_t \, e_{x_0} + (1 - \alpha_t) \, \pi}$$

The expected noisy belief P_t given the clean token x_0 equals α_t times e_{x0}, the one-hot vector of x_0, plus one minus α_t times the distribution π.

First, we need a forward process for the beliefs

The noisy belief P_t at a position fluctuates around our forward marginal:

$$\boxed{\mathbb{E}\left[P_t \mid x_0\right] = \alpha_t \, e_{x_0} + (1 - \alpha_t) \, \pi}$$

The expected noisy belief P_t given the clean token x_0 equals α_t times e_{x0}, the one-hot vector of x_0, plus one minus α_t times the distribution π.

The beliefs follow a Dirichlet distribution around this mean, with a concentration that controls how far they spread.

First, we need a forward process for the beliefs

The noisy belief P_t at a position fluctuates around our forward marginal:

$$\boxed{\mathbb{E}\left[P_t \mid x_0\right] = \alpha_t \, e_{x_0} + (1 - \alpha_t) \, \pi}$$

The expected noisy belief P_t given the clean token x_0 equals α_t times e_{x0}, the one-hot vector of x_0, plus one minus α_t times the distribution π.

The beliefs follow a Dirichlet distribution around this mean, with a concentration that controls how far they spread.

As the concentration goes to zero, every belief lands on a corner with the uniform-diffusion probabilities.

First, we need a forward process for the beliefs

The noisy belief P_t at a position fluctuates around our forward marginal:

$$\boxed{\mathbb{E}\left[P_t \mid x_0\right] = \alpha_t \, e_{x_0} + (1 - \alpha_t) \, \pi}$$

The expected noisy belief P_t given the clean token x_0 equals α_t times e_{x0}, the one-hot vector of x_0, plus one minus α_t times the distribution π.

The beliefs follow a Dirichlet distribution around this mean, with a concentration that controls how far they spread.

A belief can keep both A and G plausible while the rest of the sequence provides more evidence.

Next, we write the reverse step for beliefs

Next, we write the reverse step for beliefs

Given x_0, the reverse step has a closed form, so the model averages over the denoiser:

$$p_\theta(P_s \mid P_t) = \sum_{x_0} d_\theta(x_0 \mid P_t, t)\; q(P_s \mid P_t, x_0)$$

Next, we write the reverse step for beliefs

Given x_0, the reverse step has a closed form, so the model averages over the denoiser:

$$p_\theta(P_s \mid P_t) = \sum_{x_0} d_\theta(x_0 \mid P_t, t)\; q(P_s \mid P_t, x_0)$$

The model's reverse step $p_\theta(P_s \mid P_t)$ from the belief P_t to an earlier belief P_s equals the sum over clean tokens x_0 of the denoiser probability $d_\theta(x_0 \mid P_t, t)$ times the reverse step $q(P_s \mid P_t, x_0)$ given x_0.

Next, we write the reverse step for beliefs

Given x_0, the reverse step has a closed form, so the model averages over the denoiser:

$$p_\theta(P_s \mid P_t) = \sum_{x_0} d_\theta(x_0 \mid P_t, t)\; q(P_s \mid P_t, x_0)$$

The model's reverse step $p_\theta(P_s \mid P_t)$ from the belief P_t to an earlier belief P_s equals the sum over clean tokens x_0 of the denoiser probability $d_\theta(x_0 \mid P_t, t)$ times the reverse step $q(P_s \mid P_t, x_0)$ given x_0.

We train the denoiser with cross-entropy, whose optimum is the posterior:

$$\mathcal{L}(\theta) = \mathbb{E}\big[-\log d_\theta(x_0 \mid P_t, t)\big]$$

Next, we write the reverse step for beliefs

Given x_0, the reverse step has a closed form, so the model averages over the denoiser:

$$p_\theta(P_s \mid P_t) = \sum_{x_0} d_\theta(x_0 \mid P_t, t)\; q(P_s \mid P_t, x_0)$$

The model's reverse step $p_\theta(P_s \mid P_t)$ from the belief P_t to an earlier belief P_s equals the sum over clean tokens x_0 of the denoiser probability $d_\theta(x_0 \mid P_t, t)$ times the reverse step $q(P_s \mid P_t, x_0)$ given x_0.

We train the denoiser with cross-entropy, whose optimum is the posterior:

$$\mathcal{L}(\theta) = \mathbb{E}\big[-\log d_\theta(x_0 \mid P_t, t)\big]$$

The loss $L(\theta)$ is the expected negative log probability, negative log $d_\theta(x_0 \mid P_t, t)$, that the denoiser assigns to the clean token x_0 given the noisy belief P_t at time t.

Next, we write the reverse step for beliefs

Given x_0, the reverse step has a closed form, so the model averages over the denoiser:

$$p_\theta(P_s \mid P_t) = \sum_{x_0} d_\theta(x_0 \mid P_t, t)\; q(P_s \mid P_t, x_0)$$

The model's reverse step $p_\theta(P_s \mid P_t)$ from the belief P_t to an earlier belief P_s equals the sum over clean tokens x_0 of the denoiser probability $d_\theta(x_0 \mid P_t, t)$ times the reverse step $q(P_s \mid P_t, x_0)$ given x_0.

We train the denoiser with cross-entropy, whose optimum is the posterior:

$$\mathcal{L}(\theta) = \mathbb{E}\big[-\log d_\theta(x_0 \mid P_t, t)\big]$$

The loss $L(\theta)$ is the expected negative log probability, negative log $d_\theta(x_0 \mid P_t, t)$, that the denoiser assigns to the clean token x_0 given the noisy belief P_t at time t.

The forward marginal, the posterior average, and the cross-entropy are the same three pieces we used for MDLM.

Next, we write the reverse step for beliefs

Given x_0, the reverse step has a closed form, so the model averages over the denoiser:

$$p_\theta(P_s \mid P_t) = \sum_{x_0} d_\theta(x_0 \mid P_t, t)\; q(P_s \mid P_t, x_0)$$

The model's reverse step $p_\theta(P_s \mid P_t)$ from the belief P_t to an earlier belief P_s equals the sum over clean tokens x_0 of the denoiser probability $d_\theta(x_0 \mid P_t, t)$ times the reverse step $q(P_s \mid P_t, x_0)$ given x_0.

We train the denoiser with cross-entropy, whose optimum is the posterior:

$$\mathcal{L}(\theta) = \mathbb{E}\big[-\log d_\theta(x_0 \mid P_t, t)\big]$$

The loss $L(\theta)$ is the expected negative log probability, negative log $d_\theta(x_0 \mid P_t, t)$, that the denoiser assigns to the clean token x_0 given the noisy belief P_t at time t.

Each possible clean token proposes a reverse step, and the denoiser determines how much weight that proposal receives.

Simplex diffusion: averaging a belief update

Toy denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Let B mean that the next belief puts more than half its mass on G. Given the clean base, the four probabilities of B are (0.1, 0.2, 0.8, 0.1).

Simplex diffusion: averaging a belief update

Toy denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Let B mean that the next belief puts more than half its mass on G. Given the clean base, the four probabilities of B are (0.1, 0.2, 0.8, 0.1).

$$p_\theta(B) = (0.1)(0.1) + (0.2)(0.2) + (0.6)(0.8) + (0.1)(0.1) = 0.54$$

Simplex diffusion: averaging a belief update

Toy denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Let B mean that the next belief puts more than half its mass on G. Given the clean base, the four probabilities of B are (0.1, 0.2, 0.8, 0.1).

$$p_\theta(B) = (0.1)(0.1) + (0.2)(0.2) + (0.6)(0.8) + (0.1)(0.1) = 0.54$$

Average the four possible reverse updates with the denoiser weights. Event B gets probability 0.54.

Simplex diffusion: averaging a belief update

Toy denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Let B mean that the next belief puts more than half its mass on G. Given the clean base, the four probabilities of B are (0.1, 0.2, 0.8, 0.1).

$$p_\theta(B) = (0.1)(0.1) + (0.2)(0.2) + (0.6)(0.8) + (0.1)(0.1) = 0.54$$

Average the four possible reverse updates with the denoiser weights. Event B gets probability 0.54.

$$x_0 = \mathrm{G}: \qquad -\log d_\theta(\mathrm{G} \mid P_t, t) = -\log(0.6) \approx 0.511$$

Simplex diffusion: averaging a belief update

Toy denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Let B mean that the next belief puts more than half its mass on G. Given the clean base, the four probabilities of B are (0.1, 0.2, 0.8, 0.1).

$$p_\theta(B) = (0.1)(0.1) + (0.2)(0.2) + (0.6)(0.8) + (0.1)(0.1) = 0.54$$

Average the four possible reverse updates with the denoiser weights. Event B gets probability 0.54.

$$x_0 = \mathrm{G}: \qquad -\log d_\theta(\mathrm{G} \mid P_t, t) = -\log(0.6) \approx 0.511$$

If the clean base was G, this example contributes 0.511 nats of cross-entropy.

Simplex diffusion: averaging a belief update

Toy denoiser probabilities for A, C, G, T are (0.1, 0.2, 0.6, 0.1). Let B mean that the next belief puts more than half its mass on G. Given the clean base, the four probabilities of B are (0.1, 0.2, 0.8, 0.1).

$$p_\theta(B) = (0.1)(0.1) + (0.2)(0.2) + (0.6)(0.8) + (0.1)(0.1) = 0.54$$

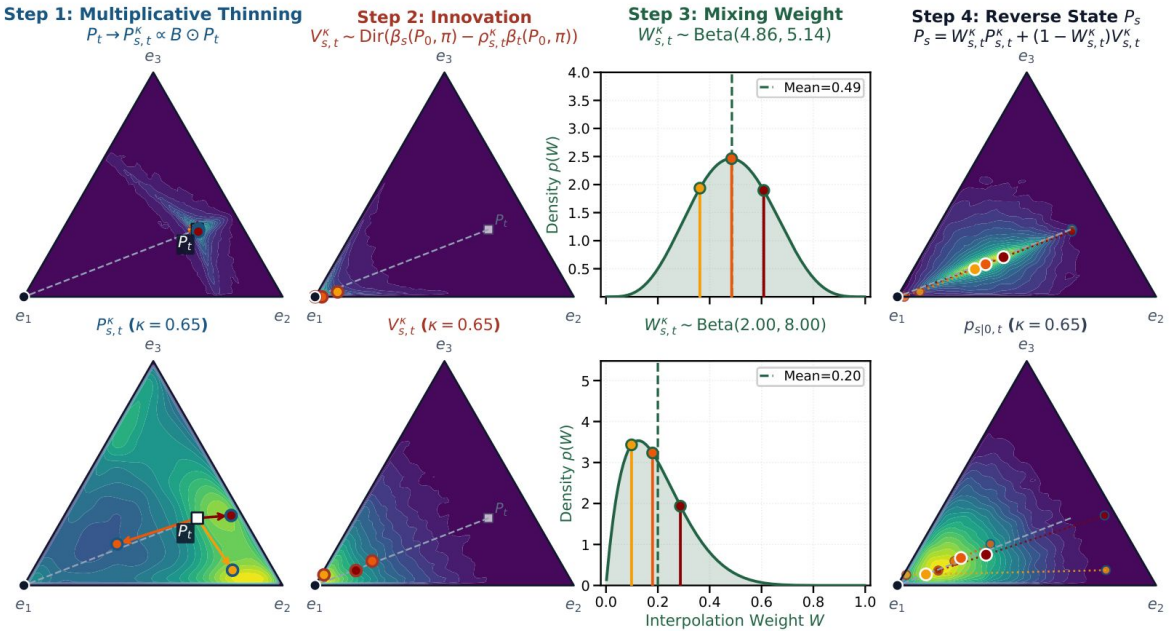
Average the four possible reverse updates with the denoiser weights. Event B gets probability 0.54.

$$x_0 = \mathrm{G}: \qquad -\log d_\theta(\mathrm{G} \mid P_t, t) = -\log(0.6) \approx 0.511$$

If the clean base was G, this example contributes 0.511 nats of cross-entropy.

The model averages over uncertainty about the clean base before it updates the noisy belief.

In one reverse step, we thin the current belief, draw an innovation, and mix the two



Deschenaux,..,De Bortoli. *arXiv* (2026)

Here is Figure 1 of the paper, on reasoning problems

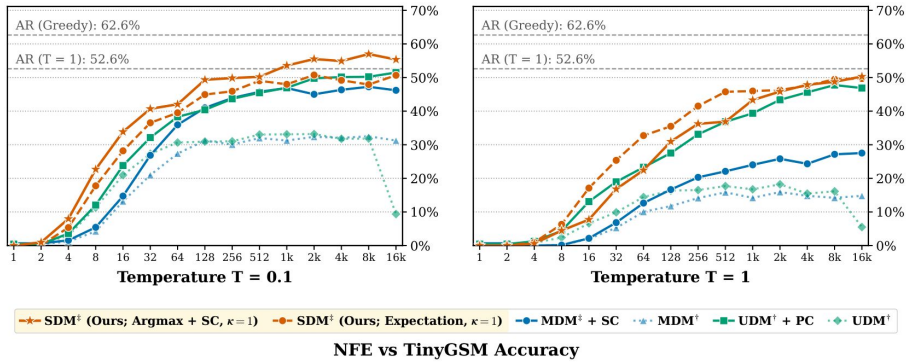


Figure 1: **SDMs outperform Discrete Diffusion on TinyGSM.** *Left:* Accuracy as a function of the number of function evaluation (NFE) at low temperature ($T = 0.1$). *Right:* Accuracy against NFEs at $T = 1$. Without Self-Conditioning (SC), SDMs with the expected embedding outperform masked diffusion with SC (MDM + SC) and uniform diffusion with predictor-corrector sampling (UDM + PC). At 512 NFE, SDMs reach 49.0% at $T = 0.1$, vs. 45.8% for MDM‡ + SC, and 45.8% at $T = 1$, vs. 36.8% for UDM† + PC. With SC, SDM (Argmax + SC) is the best diffusion variant at $T = 0.1$ from 32 NFE onwards, and solves 57% of problems with 8k NFEs. Autoregressive models with greedy decoding is best (62.6%). All SDMs use churn $\kappa = 1$ and the adaptive sampling time grid (Section J.1), and we compare with the best baseline configurations (Section 5.2). $^{\dagger}/^{\ddagger}$: trained with the uniform/adaptive time sampler. The accuracy drop for UDM at 16k is consistent with typical floating-point rounding errors, which can accumulate at very small step sizes (Karras et al., 2022).

Deschenaux,..,De Bortoli. *arXiv* (2026)

Accuracy on TinyGSM rises with the number of function evaluations

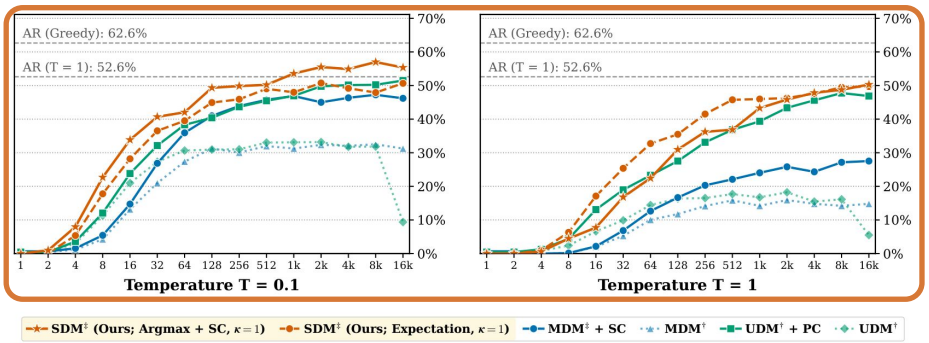


Figure 1: **SDMs outperform Discrete Diffusion on TinyGSM.** *Left:* Accuracy as a function of the number of function evaluation (NFE) at low temperature ($T = 0.1$). *Right:* Accuracy against NFEs at $T = 1$. Without Self-Conditioning (SC), SDMs with the expected embedding outperform masked diffusion with SC (MDM + SC) and uniform diffusion with predictor-corrector sampling (UDM + PC). At 512 NFE, SDMs reach 49.0% at $T = 0.1$, vs. 45.8% for MDM‡ + SC, and 45.8% at $T = 1$, vs. 36.8% for UDM† + PC. With SC, SDM (Argmax + SC) is the best diffusion variant at $T = 0.1$ from 32 NFE onwards, and solves 57% of problems with 8k NFEs. Autoregressive models with greedy decoding is best (62.6%). All SDMs use churn $\kappa = 1$ and the adaptive sampling time grid (Section J.1), and we compare with the best baseline configurations (Section 5.2). $^{\dagger}/^{\ddagger}$: trained with the uniform/adaptive time sampler. The accuracy drop for UDM at 16k is consistent with typical floating-point rounding errors, which can accumulate at very small step sizes (Karras et al., 2022).

Deschenaux,..,De Bortoli. *arXiv* (2026)

At 512 evaluations, simplex diffusion solves 49.0% against 45.8% for masked diffusion

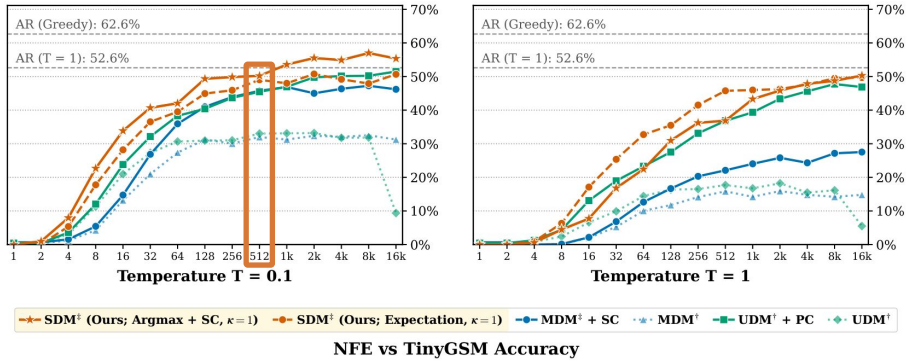


Figure 1: **SDMs outperform Discrete Diffusion on TinyGSM.** *Left:* Accuracy as a function of the number of function evaluation (NFE) at low temperature ($T = 0.1$). *Right:* Accuracy against NFEs at $T = 1$. Without Self-Conditioning (SC), SDMs with the expected embedding outperform masked diffusion with SC (MDM + SC) and uniform diffusion with predictor-corrector sampling (UDM + PC). At 512 NFE, SDMs reach 49.0% at $T = 0.1$, vs. 45.8% for MDM‡ + SC, and 45.8% at $T = 1$, vs. 36.8% for UDM† + PC. With SC, SDM (Argmax + SC) is the best diffusion variant at $T = 0.1$ from 32 NFE onwards, and solves 57% of problems with 8k NFEs. Autoregressive models with greedy decoding is best (62.6%). All SDMs use churn $\kappa = 1$ and the adaptive sampling time grid (Section J.1), and we compare with the best baseline configurations (Section 5.2). $^{\dagger}/^{\ddagger}$: trained with the uniform/adaptive time sampler. The accuracy drop for UDM at 16k is consistent with typical floating-point rounding errors, which can accumulate at very small step sizes (Karras et al., 2022).

Deschenaux,..,De Bortoli. *arXiv* (2026)

With self-conditioning it reaches 57% at 8k evaluations

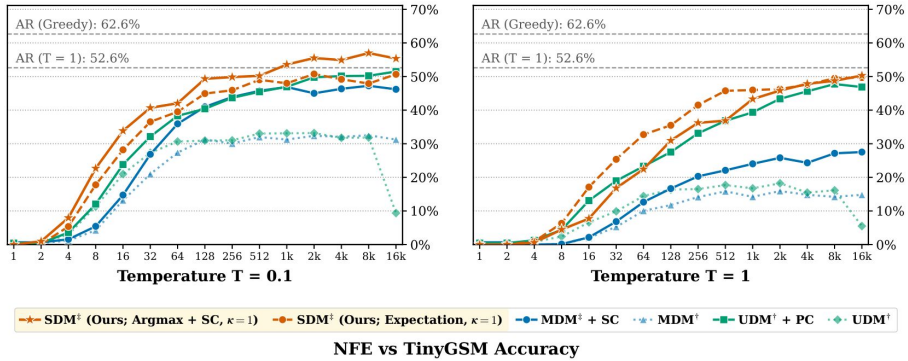


Figure 1: **SDMs outperform Discrete Diffusion on TinyGSM.** *Left:* Accuracy as a function of the number of function evaluation (NFE) at low temperature ($T = 0.1$). *Right:* Accuracy against NFEs at $T = 1$. Without Self-Conditioning (SC), SDMs with the expected embedding outperform masked diffusion with SC (MDM + SC) and uniform diffusion with predictor-corrector sampling (UDM + PC). At 512 NFE, SDMs reach 49.0% at $T = 0.1$, vs. 45.8% for MDM‡ + SC, and 45.8% at $T = 1$, vs. 36.8% for UDM† + PC. With SC, SDM (Argmax + SC) is the best diffusion variant at $T = 0.1$ from 32 NFE onwards, and solves 57% of problems with 8k NFEs. Autoregressive models with greedy decoding is best (62.6%). All SDMs use churn $\kappa = 1$ and the adaptive sampling time grid (Section J.1), and we compare with the best baseline configurations (Section 5.2). $^{\dagger}/^{\ddagger}$: trained with the uniform/adaptive time sampler. The accuracy drop for UDM at 16k is consistent with typical floating-point rounding errors, which can accumulate at very small step sizes (Karras et al., 2022).

Deschenaux,..,De Bortoli. *arXiv* (2026)

Greedy autoregressive decoding is still ahead, at 62.6%

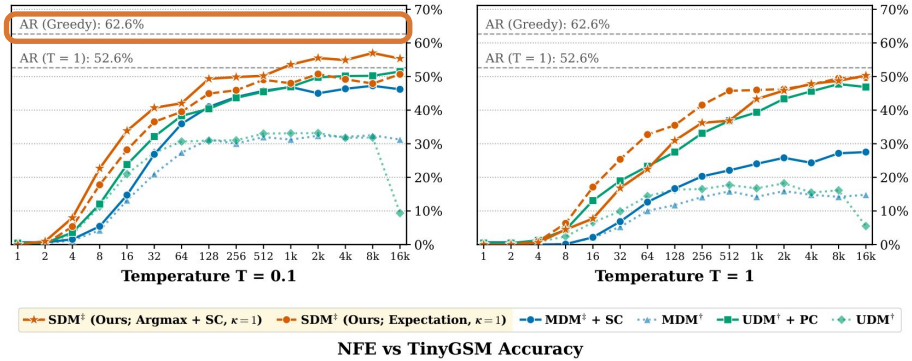


Figure 1: **SDMs outperform Discrete Diffusion on TinyGSM.** *Left:* Accuracy as a function of the number of function evaluation (NFE) at low temperature ($T = 0.1$). *Right:* Accuracy against NFEs at $T = 1$. Without Self-Conditioning (SC), SDMs with the expected embedding outperform masked diffusion with SC (MDM + SC) and uniform diffusion with predictor-corrector sampling (UDM + PC). At 512 NFE, SDMs reach 49.0% at $T = 0.1$, vs. 45.8% for MDM‡ + SC, and 45.8% at $T = 1$, vs. 36.8% for UDM† + PC. With SC, SDM (Argmax + SC) is the best diffusion variant at $T = 0.1$ from 32 NFE onwards, and solves 57% of problems with 8k NFEs. Autoregressive models with greedy decoding is best (62.6%). All SDMs use churn $\kappa = 1$ and the adaptive sampling time grid (Section J.1), and we compare with the best baseline configurations (Section 5.2). $^{\dagger}/^{\ddagger}$: trained with the uniform/adaptive time sampler. The accuracy drop for UDM at 16k is consistent with typical floating-point rounding errors, which can accumulate at very small step sizes (Karras et al., 2022).

Deschenaux,..,De Bortoli. *arXiv* (2026)

With beliefs, we carry the denoiser's uncertainty from one step to the next

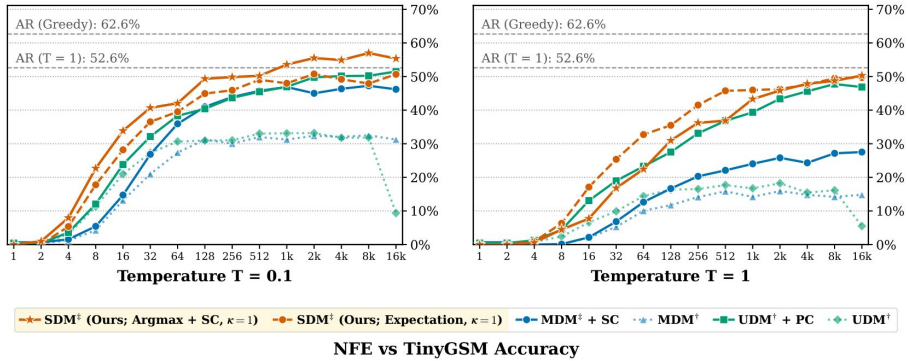


Figure 1: **SDMs outperform Discrete Diffusion on TinyGSM.** *Left:* Accuracy as a function of the number of function evaluation (NFE) at low temperature ($T = 0.1$). *Right:* Accuracy against NFEs at $T = 1$. Without Self-Conditioning (SC), SDMs with the expected embedding outperform masked diffusion with SC (MDM + SC) and uniform diffusion with predictor-corrector sampling (UDM + PC). At 512 NFE, SDMs reach 49.0% at $T = 0.1$, vs. 45.8% for MDM‡ + SC, and 45.8% at $T = 1$, vs. 36.8% for UDM† + PC. With SC, SDM (Argmax + SC) is the best diffusion variant at $T = 0.1$ from 32 NFE onwards, and solves 57% of problems with 8k NFEs. Autoregressive models with greedy decoding is best (62.6%). All SDMs use churn $\kappa = 1$ and the adaptive sampling time grid (Section J.1), and we compare with the best baseline configurations (Section 5.2). $^{\dagger}/^{\ddagger}$: trained with the uniform/adaptive time sampler. The accuracy drop for UDM at 16k is consistent with typical floating-point rounding errors, which can accumulate at very small step sizes (Karras et al., 2022).

Deschenaux,..,De Bortoli. *arXiv* (2026)

Let’s write the whole model for DNA

```
MASK = 4                              # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                   # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)             # [B, L, 5]
    logits[..., MASK] = -1e9          # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                       # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

Our alphabet is four bases and one mask

- The mask is token 4, the extra absorbing state.

```
MASK = 4                                  # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                       # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)                 # [B, L, 5]
    logits[..., MASK] = -1e9              # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                           # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

To corrupt, we sample the forward marginal directly

- The mask is token 4, the extra absorbing state.
- Each position survives with probability $\alpha_t = 1 - t$, independently.

```python
MASK = 4                                  # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                       # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)                 # [B, L, 5]
    logits[..., MASK] = -1e9              # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                           # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

To corrupt, we sample the forward marginal directly

- The mask is token 4, the extra absorbing state.
- Each position survives with probability $\alpha_t = 1 - t$, independently.
- By fact 2, we jump to any noise level in one line, with no trajectory.

```
MASK = 4                                # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                     # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)               # [B, L, 5]
    logits[..., MASK] = -1e9            # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                         # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

To train, we sample one time and one corruption

- We sample t uniformly and corrupt the clean batch once.

```
MASK = 4                                # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                     # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)               # [B, L, 5]
    logits[..., MASK] = -1e9            # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                         # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

The denoiser predicts the clean token at every position

- We sample t uniformly and corrupt the clean batch once.
- The network reads the whole noisy sequence and returns five logits per position.

```
MASK = 4                                # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                     # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)               # [B, L, 5]
    logits[..., MASK] = -1e9            # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                         # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

The denoiser never predicts a mask

- We sample t uniformly and corrupt the clean batch once.
- The network reads the whole noisy sequence and returns five logits per position.
- This is the zero-mask rule. The posterior over clean tokens has no mass on m.

```
MASK = 4                                # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                     # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)               # [B, L, 5]
    logits[..., MASK] = -1e9            # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                         # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

We score the correct clean token with cross-entropy

- The rate error at a masked position reduced to this one term.

```
MASK = 4                                # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                     # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)               # [B, L, 5]
    logits[..., MASK] = -1e9            # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                         # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

The weight is the reveal rate w(t)

- The rate error at a masked position reduced to this one term.
- With $\alpha_t = 1 - t$, the weight $-\dot{\alpha}_t/(1 - \alpha_t)$ is exactly $1/t$.

```
MASK = 4                              # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                   # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)             # [B, L, 5]
    logits[..., MASK] = -1e9          # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                       # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

Only masked positions contribute

- The rate error at a masked position reduced to this one term.
- With $\alpha_t = 1 - t$, the weight $-\dot{\alpha}_t/(1 - \alpha_t)$ is exactly $1/t$.
- Visible positions have zero rate in both processes, so their loss is zero.

```python
MASK = 4                                # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                     # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)               # [B, L, 5]
    logits[..., MASK] = -1e9            # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                         # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

Generation starts where the forward process ends

- All masks is the stationary distribution of the masking rates.

```
MASK = 4                                # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                     # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)               # [B, L, 5]
    logits[..., MASK] = -1e9            # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                         # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

At each step, we draw a clean token from the denoiser

- All masks is the stationary distribution of the masking rates.
- This sample is the posterior average turned into one reverse step.

```python
MASK = 4                                # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                     # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)               # [B, L, 5]
    logits[..., MASK] = -1e9            # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                         # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

Each masked position reveals with probability (t – s)/t

- All masks is the stationary distribution of the masking rates.
- This sample is the posterior average turned into one reverse step.
- That is the reveal rate w(t) integrated over the step, α_s/α_t apart from one.

```python
MASK = 4                                # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                     # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)               # [B, L, 5]
    logits[..., MASK] = -1e9            # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                         # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

Revealed tokens never change again

- This is carry-over unmasking, which we derived from the rates being zero.

```
MASK = 4                              # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                   # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)             # [B, L, 5]
    logits[..., MASK] = -1e9          # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                       # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

Thirty lines hold the entire framework for masking

- This is carry-over unmasking, which we derived from the rates being zero.

```
MASK = 4                              # A, C, G, T, then the mask

def corrupt(x0, t):
    alpha = 1.0 - t                   # linear schedule
    keep = torch.rand(x0.shape, device=DEV) < alpha
    return torch.where(keep, x0, MASK)

def loss(model, x0):
    t = torch.rand(x0.shape[0], device=DEV)
    zt = corrupt(x0, t[:, None])
    logits = model(zt, t)             # [B, L, 5]
    logits[..., MASK] = -1e9          # no mass on the mask
    ce = F.cross_entropy(logits.transpose(1, 2), x0,
                         reduction='none')
    w = 1.0 / t                       # -alpha_dot/(1 - alpha)
    return (w[:, None] * ce * (zt == MASK)).sum(-1).mean()

@torch.no_grad()
def sample(model, B, L, steps=32):
    z = torch.full((B, L), MASK, device=DEV)
    ts = torch.linspace(1.0, 1e-3, steps + 1, device=DEV)
    for t, s in zip(ts[:-1], ts[1:]):
        logits = model(z, t.expand(B))
        logits[..., MASK] = -1e9
        x0 = Categorical(logits=logits).sample()
        unmask = torch.rand(z.shape, device=DEV) < (t - s) / t
        z = torch.where((z == MASK) & unmask, x0, z)
    return z
```

Now adapt these thirty lines to the other models

Now adapt these thirty lines to the other models

For uniform noise, we corrupt with a replacement token instead of the mask, and we use all K – 1 rates in the reverse step.

Now adapt these thirty lines to the other models

For uniform noise, we corrupt with a replacement token instead of the mask, and we use all K – 1 rates in the reverse step.

For block diffusion, the loop runs over blocks, and the denoiser also reads the clean prefix.

Now adapt these thirty lines to the other models

For uniform noise, we corrupt with a replacement token instead of the mask, and we use all $K - 1$ rates in the reverse step.

For block diffusion, the loop runs over blocks, and the denoiser also reads the clean prefix.

In the rest of this lecture, we change how we choose the next position, how we weight the loss, and what we condition on.

Let’s start with the choice of position

For uniform noise, we corrupt with a replacement token instead of the mask, and we use all K – 1 rates in the reverse step.

For block diffusion, the loop runs over blocks, and the denoiser also reads the clean prefix.

In the rest of this lecture, we change how we choose the next position, how we weight the loss, and what we condition on.

If the model is more confident at some positions, why reveal them in a random order?

P2: Path Planning

Path Planning for Diffusion Language Model Sampling

Fred Zhangzhi Peng[1,*,‡], **Zachary Bezemek**[1,*], **Sawan Patel**[2], **Jarrid Rector-Brooks**[3,4], **Sherwood Yao**[2], **Avishek Joey Bose**[3,5], **Alexander Tong**[3,4,6,†,‡], **Pranam Chatterjee**[7,†,‡]

[1]Duke University, [2]Atom Bioworks, [3]Mila – Québec AI Institute,
[4]Université de Montréal, [5]The University of Oxford, [6]Aithyra, [7]University of Pennsylvania
[*]Equal contribution, [†]Equal co-supervision
[‡]Corresponding authors: zp70@duke.edu, atong@aithyra.at, pranam@seas.upenn.edu

Peng,..,Chatterjee. *TMLR* (2026)

Recall that we reveal the masked positions in a random order

Recall that we reveal the masked positions in a random order

In our sampler, every masked position has the same probability of being revealed, whatever the denoiser predicts there.

Recall that we reveal the masked positions in a random order

In our sampler, every masked position has the same probability of being revealed, whatever the denoiser predicts there.

Two changes follow from the math we already have. Pick the positions we are most sure about, and allow a corrector step to revisit them.

Recall that we reveal the masked positions in a random order

In our sampler, every masked position has the same probability of being revealed, whatever the denoiser predicts there.

Two changes follow from the math we already have. Pick the positions we are most sure about, and allow a corrector step to revisit them.

The corrector rates we derived leave p_t unchanged: $C_t = R_t + \hat{R}_t$

Recall that we reveal the masked positions in a random order

In our sampler, every masked position has the same probability of being revealed, whatever the denoiser predicts there.

Two changes follow from the math we already have. Pick the positions we are most sure about, and allow a corrector step to revisit them.

The corrector rates we derived leave p_t unchanged: $C_t = R_t + \hat{R}_t$

The corrector rate matrix C_t equals the forward rates R_t plus the reverse rates $\hat{R}_t$ at the same time t.

Recall that we reveal the masked positions in a random order

In our sampler, every masked position has the same probability of being revealed, whatever the denoiser predicts there.

Two changes follow from the math we already have. Pick the positions we are most sure about, and allow a corrector step to revisit them.

The corrector rates we derived leave p_t unchanged: $C_t = R_t + \hat{R}_t$

The corrector rate matrix C_t equals the forward rates R_t plus the reverse rates $\hat{R}_t$ at the same time t.

Masking a token again and revealing it is one such step.

Here is one sampling step of P2 in code

```
def p2_step(model, z, t, s, n_unmask, n_remask):
    logits = model(z, t)
    logits[..., MASK] = -1e9
    p = logits.softmax(-1)
    x0 = Categorical(probs=p).sample()

    # planner score: how sure the denoiser is here
    score = p.max(-1).values                # or -entropy, or a
                                            # separate planner
    masked = (z == MASK)
    pick = topk_where(score, masked, n_unmask)
    z = torch.where(pick, x0, z)            # unmask the best

    # corrector: send the least certain visible tokens back
    back = topk_where(-score, ~masked, n_remask)
    z = torch.where(back, MASK, z)          # remask the worst
    return z
```

First, look at where the denoiser is confident

- From one forward pass, we get a distribution at every masked position.

```
def p2_step(model, z, t, s, n_unmask, n_remask):
    logits = model(z, t)
    logits[..., MASK] = -1e9
    p = logits.softmax(-1)
    x0 = Categorical(probs=p).sample()

    # planner score: how sure the denoiser is here
    score = p.max(-1).values                  # or -entropy, or a
                                              # separate planner
    masked = (z == MASK)
    pick = topk_where(score, masked, n_unmask)
    z = torch.where(pick, x0, z)              # unmask the best

    # corrector: send the least certain visible tokens back
    back = topk_where(-score, ~masked, n_remask)
    z = torch.where(back, MASK, z)            # remask the worst
    return z
```

Next, rank the masked positions by the planner score

- From one forward pass, we get a distribution at every masked position.
- In P2, we score positions by confidence, by entropy, or with a separate planner.

```
def p2_step(model, z, t, s, n_unmask, n_remask):
    logits = model(z, t)
    logits[..., MASK] = -1e9
    p = logits.softmax(-1)
    x0 = Categorical(probs=p).sample()

    # planner score: how sure the denoiser is here
    score = p.max(-1).values                    # or -entropy, or a
                                                # separate planner
    masked = (z == MASK)
    pick = topk_where(score, masked, n_unmask)
    z = torch.where(pick, x0, z)                # unmask the best

    # corrector: send the least certain visible tokens back
    back = topk_where(-score, ~masked, n_remask)
    z = torch.where(back, MASK, z)              # remask the worst
    return z
```

We reveal the positions we are most sure about

- The reverse rates are unchanged, and only the order of reveals changes.

```
def p2_step(model, z, t, s, n_unmask, n_remask):
    logits = model(z, t)
    logits[..., MASK] = -1e9
    p = logits.softmax(-1)
    x0 = Categorical(probs=p).sample()

    # planner score: how sure the denoiser is here
    score = p.max(-1).values                  # or -entropy, or a
                                              # separate planner
    masked = (z == MASK)
    pick = topk_where(score, masked, n_unmask)
    z = torch.where(pick, x0, z)              # unmask the best

    # corrector: send the least certain visible tokens back
    back = topk_where(-score, ~masked, n_remask)
    z = torch.where(back, MASK, z)            # remask the worst
    return z
```

In a corrector step, uncertain tokens go back to the mask

- The reverse rates are unchanged, and only the order of reveals changes.
- Masking a token again and revealing it is a corrector step, and p_t is stationary for it.

```python
def p2_step(model, z, t, s, n_unmask, n_remask):
    logits = model(z, t)
    logits[..., MASK] = -1e9
    p = logits.softmax(-1)
    x0 = Categorical(probs=p).sample()

    # planner score: how sure the denoiser is here
    score = p.max(-1).values                # or -entropy, or a
                                            # separate planner
    masked = (z == MASK)
    pick = topk_where(score, masked, n_unmask)
    z = torch.where(pick, x0, z)            # unmask the best

    # corrector: send the least certain visible tokens back
    back = topk_where(-score, ~masked, n_remask)
    z = torch.where(back, MASK, z)          # remask the worst
    return z
```

So we can plan and remask without retraining the model

- The reverse rates are unchanged, and only the order of reveals changes.
- Masking a token again and revealing it is a corrector step, and p_t is stationary for it.

```python
def p2_step(model, z, t, s, n_unmask, n_remask):
    logits = model(z, t)
    logits[..., MASK] = -1e9
    p = logits.softmax(-1)
    x0 = Categorical(probs=p).sample()

    # planner score: how sure the denoiser is here
    score = p.max(-1).values                # or -entropy, or a
                                            # separate planner
    masked = (z == MASK)
    pick = topk_where(score, masked, n_unmask)
    z = torch.where(pick, x0, z)            # unmask the best

    # corrector: send the least certain visible tokens back
    back = topk_where(-score, ~masked, n_remask)
    z = torch.where(back, MASK, z)          # remask the worst
    return z
```

P2: a reveal and a correction

Toy state A [MASK] G [MASK]. The planner scores positions 1 through 4 as (0.55, 0.90, 0.80, 0.60). Reveal one masked position, then remask one previously visible position.

P2: a reveal and a correction

Toy state A [MASK] G [MASK]. The planner scores positions 1 through 4 as (0.55, 0.90, 0.80, 0.60). Reveal one masked position, then remask one previously visible position.

$$i_{\text{reveal}} = \arg\max_{i \in \{2,4\}} s_i = 2, \qquad z' = \text{A C G [MASK]}$$

P2: a reveal and a correction

Toy state A [MASK] G [MASK]. The planner scores positions 1 through 4 as (0.55, 0.90, 0.80, 0.60). Reveal one masked position, then remask one previously visible position.

$$i_{\text{reveal}} = \arg\max_{i \in \{2,4\}} s_i = 2, \qquad z' = \text{A C G [MASK]}$$

Position 2 wins with score 0.90. Suppose the denoiser samples C there.

P2: a reveal and a correction

Toy state A [MASK] G [MASK]. The planner scores positions 1 through 4 as (0.55, 0.90, 0.80, 0.60). Reveal one masked position, then remask one previously visible position.

$$i_{\text{reveal}} = \arg\max_{i \in \{2,4\}} s_i = 2, \qquad z' = \text{A C G [MASK]}$$

Position 2 wins with score 0.90. Suppose the denoiser samples C there.

$$i_{\text{remask}} = \arg\min_{i \in \{1,3\}} s_i = 1, \qquad z'' = \text{[MASK] C G [MASK]}$$

P2: a reveal and a correction

Toy state A [MASK] G [MASK]. The planner scores positions 1 through 4 as (0.55, 0.90, 0.80, 0.60). Reveal one masked position, then remask one previously visible position.

$$i_{\text{reveal}} = \arg\max_{i \in \{2,4\}} s_i = 2, \qquad z' = \text{A C G [MASK]}$$

Position 2 wins with score 0.90. Suppose the denoiser samples C there.

$$i_{\text{remask}} = \arg\min_{i \in \{1,3\}} s_i = 1, \qquad z'' = \text{[MASK] C G [MASK]}$$

Among the two previously visible positions, position 1 has the lower score, so it becomes a mask.

P2: a reveal and a correction

Toy state A [MASK] G [MASK]. The planner scores positions 1 through 4 as (0.55, 0.90, 0.80, 0.60). Reveal one masked position, then remask one previously visible position.

$$i_{\text{reveal}} = \arg\max_{i \in \{2,4\}} s_i = 2, \qquad z' = \text{A C G [MASK]}$$

Position 2 wins with score 0.90. Suppose the denoiser samples C there.

$$i_{\text{remask}} = \arg\min_{i \in \{1,3\}} s_i = 1, \qquad z'' = \text{[MASK] C G [MASK]}$$

Among the two previously visible positions, position 1 has the lower score, so it becomes a mask.

The planner chooses where to spend the next denoising step, including where to reconsider a decision.

We select the positions with a planner, and we remask to correct earlier choices

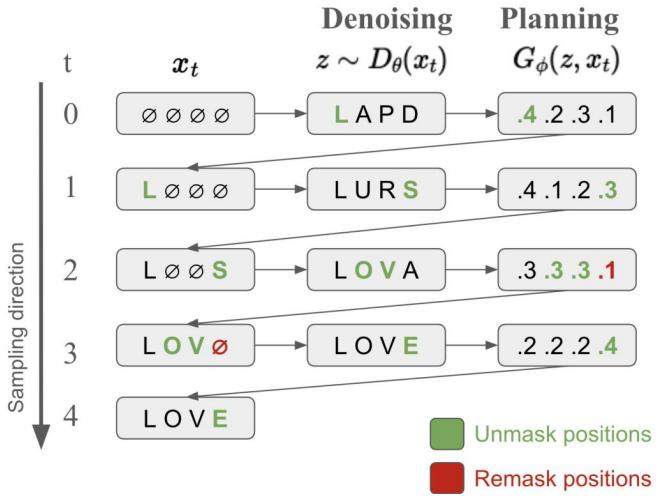


Figure 1: Illustration of P2 sampling (Algorithm 1). At each step, the denoiser D_θ predicts z, and the planner G_ϕ selects positions to unmask (green) and remask (red).

We can sample any MDLM checkpoint this way.

Peng,..,Chatterjee. *TMLR* (2026)

Here is the protein benchmark in Table 2 of the paper

Table 2: Protein sequence generation benchmark. We evaluate structure quality via pLDDT, pTM, and pAE, and diversity via token entropy and sequence uniqueness. Foldability is the percentage of sequences satisfying pLDDT > 80, pTM > 0.7, and pAE < 10. See Section F.1 for setup and Table S4 for model size ablations.

Model	pLDDT↑	pTM↑	pAE↓	Foldability (%)↑	Entropy↑	Diversity (%)↑
EvoDiff	31.84	0.21	24.76	0.43	**4.05**	93.19
ESM3	34.13	0.23	24.65	1.50	3.99	**93.44**
ProGen2	49.38	0.28	23.38	4.48	2.55	89.31
DPLM	80.23	0.65	12.07	48.14	3.14	92.80
DPLM + P2-train (ours)	**83.45**	**0.72**	**10.15**	**58.86**	3.35	92.69

Peng,..,Chatterjee. *TMLR* (2026)

A masked diffusion protein model folds 48.1% of its samples

Table 2: Protein sequence generation benchmark. We evaluate structure quality via pLDDT, pTM, and pAE, and diversity via token entropy and sequence uniqueness. Foldability is the percentage of sequences satisfying pLDDT > 80, pTM > 0.7, and pAE < 10. See Section F.1 for setup and Table S4 for model size ablations.

Model	pLDDT↑	pTM↑	pAE↓	Foldability (%)↑	Entropy↑	Diversity (%)↑
EvoDiff	31.84	0.21	24.76	0.43	**4.05**	93.19
ESM3	34.13	0.23	24.65	1.50	3.99	**93.44**
ProGen2	49.38	0.28	23.38	4.48	2.55	89.31
DPLM	80.23	0.65	12.07	48.14	3.14	92.80
DPLM + P2-train (ours)	**83.45**	**0.72**	**10.15**	**58.86**	3.35	92.69

Peng,..,Chatterjee. *TMLR* (2026)

When we plan the path, that rises to 58.9%, with pLDDT from 80.2 to 83.5

Table 2: Protein sequence generation benchmark. We evaluate structure quality via pLDDT, pTM, and pAE, and diversity via token entropy and sequence uniqueness. Foldability is the percentage of sequences satisfying pLDDT > 80, pTM > 0.7, and pAE < 10. See Section F.1 for setup and Table S4 for model size ablations.

Model	pLDDT↑	pTM↑	pAE↓	Foldability (%)↑	Entropy↑	Diversity (%)↑
EvoDiff	31.84	0.21	24.76	0.43	**4.05**	93.19
ESM3	34.13	0.23	24.65	1.50	3.99	**93.44**
ProGen2	49.38	0.28	23.38	4.48	2.55	89.31
DPLM	80.23	0.65	12.07	48.14	3.14	92.80
DPLM + P2-train (ours)	**83.45**	**0.72**	**10.15**	**58.86**	3.35	92.69

Peng,..,Chatterjee. *TMLR* (2026)

Only the order of reveals changed, with the same weights

Table 2: Protein sequence generation benchmark. We evaluate structure quality via pLDDT, pTM, and pAE, and diversity via token entropy and sequence uniqueness. Foldability is the percentage of sequences satisfying pLDDT > 80, pTM > 0.7, and pAE < 10. See Section F.1 for setup and Table S4 for model size ablations.

Model	pLDDT↑	pTM↑	pAE↓	Foldability (%)↑	Entropy↑	Diversity (%)↑
EvoDiff	31.84	0.21	24.76	0.43	**4.05**	93.19
ESM3	34.13	0.23	24.65	1.50	3.99	**93.44**
ProGen2	49.38	0.28	23.38	4.48	2.55	89.31
DPLM	80.23	0.65	12.07	48.14	3.14	92.80
DPLM + P2-train (ours)	**83.45**	**0.72**	**10.15**	**58.86**	3.35	92.69

Peng,..,Chatterjee. *TMLR* (2026)

If we change the order of generation, shouldn't we change training too?

PAPL: Planner-Aware Path Learning

PLANNER AWARE PATH LEARNING IN DIFFUSION LANGUAGE MODELS TRAINING

Fred Zhangzhi Peng[1,‡,*], **Zachary Bezemek**[1,‡,*], **Jarrid Rector-Brooks**[2,3,4], **Shuibai Zhang**[5], **Anru R. Zhang**[1], **Michael Bronstein**[6,7], **Alexander Tong**[7†], **Avishek Joey Bose**[2,6,8†]

[1] Duke University [2] Mila [3] Université de Montréal [4] California Institute of Technology
[5] University of Wisconsin–Madison [6] University of Oxford [7] AITHYRA
[8] Imperial College London [‡] Equal contribution [†] Equal advising

Peng,..,Bose. *ICLR* (2026)

Next, we bring the planner into training

Next, we bring the planner into training

MDLM’s loss weights every masked position equally, because we reveal them in a uniform random order.

Next, we bring the planner into training

MDLM’s loss weights every masked position equally, because we reveal them in a uniform random order.

Once we choose the order with a planner, the states we actually visit at time t are no longer uniform.

Next, we bring the planner into training

MDLM's loss weights every masked position equally, because we reveal them in a uniform random order.

Once we choose the order with a planner, the states we actually visit at time t are no longer uniform.

In PAPL, we put the planner's weights inside the rate loss:

$$\mathcal{L}_{\mathrm{PAPL}} = \int_0^1 w(t)\, \mathbb{E}\Big[\sum_{i:\, Z_t^i = m} \lambda_i(Z_t) \left(-\log d_\theta^i(x_0^i \mid Z_t, t) \right) \Big] dt$$

Next, we bring the planner into training

MDLM's loss weights every masked position equally, because we reveal them in a uniform random order.

Once we choose the order with a planner, the states we actually visit at time t are no longer uniform.

In PAPL, we put the planner's weights inside the rate loss:

$$\mathcal{L}_{\mathrm{PAPL}} = \int_0^1 w(t)\, \mathbb{E}\Big[\sum_{i:\, Z_t^i = m} \lambda_i(Z_t) \left(-\log d_\theta^i(x_0^i \mid Z_t, t) \right) \Big] dt$$

The PAPL loss L_{PAPL} equals the integral over time of the weight w(t) times the expected sum over masked positions i of the planner weight $\lambda_i(Z_t)$ times negative log $d^i_\theta(x_0{}^i \mid Z_t, t)$, the negative log probability of the clean token $x_0{}^i$.

Next, we bring the planner into training

MDLM's loss weights every masked position equally, because we reveal them in a uniform random order.

Once we choose the order with a planner, the states we actually visit at time t are no longer uniform.

In PAPL, we put the planner's weights inside the rate loss:

$$\mathcal{L}_{\mathrm{PAPL}} = \int_0^1 w(t)\, \mathbb{E}\Big[\sum_{i:\, Z_t^i = m} \lambda_i(Z_t) \left(-\log d_\theta^i(x_0^i \mid Z_t, t) \right) \Big] dt$$

The PAPL loss L_{PAPL} equals the integral over time of the weight w(t) times the expected sum over masked positions i of the planner weight $\lambda_i(Z_t)$ times negative log $d^i_\theta(x_0{}^i \mid Z_t, t)$, the negative log probability of the clean token $x_0{}^i$.

With uniform weights $\lambda_i = 1$, this is the MDLM loss we derived.

Next, we bring the planner into training

MDLM's loss weights every masked position equally, because we reveal them in a uniform random order.

Once we choose the order with a planner, the states we actually visit at time t are no longer uniform.

In PAPL, we put the planner's weights inside the rate loss:

$$\mathcal{L}_{\mathrm{PAPL}} = \int_0^1 w(t)\, \mathbb{E}\Big[\sum_{i:\, Z_t^i = m} \lambda_i(Z_t) \left(-\log d_\theta^i(x_0^i \mid Z_t, t) \right) \Big] dt$$

The PAPL loss L_{PAPL} equals the integral over time of the weight w(t) times the expected sum over masked positions i of the planner weight $\lambda_i(Z_t)$ times negative log $d^i_\theta(x_0{}^i \mid Z_t, t)$, the negative log probability of the clean token $x_0{}^i$.

If a planner reveals a position often, mistakes at that position receive more weight during training.

We still need the weights in this loss

We still need the weights in this loss

In PAPL, we derive λ_i from the planner's distribution over which position to reveal next.

We still need the weights in this loss

In PAPL, we derive λ_i from the planner's distribution over which position to reveal next.

In training, we then put more weight on the positions that we are likely to reveal.

We still need the weights in this loss

In PAPL, we derive λ_i from the planner's distribution over which position to reveal next.

In training, we then put more weight on the positions that we are likely to reveal.

The five facts still hold, with the same rates, the same marginals, and the same bound with a different weighting.

PAPL: the planner changes the training weight

Toy state A [MASK] G [MASK], with target C and T at the masks. The denoiser gives them probabilities 0.5 and 0.25. Compare equal weights with planner weights 0.8 and 0.2 at a fixed time.

PAPL: the planner changes the training weight

Toy state A [MASK] G [MASK], with target C and T at the masks. The denoiser gives them probabilities 0.5 and 0.25. Compare equal weights with planner weights 0.8 and 0.2 at a fixed time.

$$\ell_{\text{equal}} = \tfrac{1}{2}[-\log(0.5)] + \tfrac{1}{2}[-\log(0.25)] \approx 1.040$$

PAPL: the planner changes the training weight

Toy state A [MASK] G [MASK], with target C and T at the masks. The denoiser gives them probabilities 0.5 and 0.25. Compare equal weights with planner weights 0.8 and 0.2 at a fixed time.

$$\ell_{\text{equal}} = \tfrac{1}{2}[-\log(0.5)] + \tfrac{1}{2}[-\log(0.25)] \approx 1.040$$

With normalized equal weights, the two masked positions contribute equally to the average loss.

PAPL: the planner changes the training weight

Toy state A [MASK] G [MASK], with target C and T at the masks. The denoiser gives them probabilities 0.5 and 0.25. Compare equal weights with planner weights 0.8 and 0.2 at a fixed time.

$$\ell_{\text{equal}} = \tfrac{1}{2}[-\log(0.5)] + \tfrac{1}{2}[-\log(0.25)] \approx 1.040$$

With normalized equal weights, the two masked positions contribute equally to the average loss.

$$\ell_{\text{planner}} = 0.8[-\log(0.5)] + 0.2[-\log(0.25)] \approx 0.832$$

PAPL: the planner changes the training weight

Toy state A [MASK] G [MASK], with target C and T at the masks. The denoiser gives them probabilities 0.5 and 0.25. Compare equal weights with planner weights 0.8 and 0.2 at a fixed time.

$$\ell_{\text{equal}} = \tfrac{1}{2}[-\log(0.5)] + \tfrac{1}{2}[-\log(0.25)] \approx 1.040$$

With normalized equal weights, the two masked positions contribute equally to the average loss.

$$\ell_{\text{planner}} = 0.8[-\log(0.5)] + 0.2[-\log(0.25)] \approx 0.832$$

The planner puts four times as much training weight on the C position. A common time factor is omitted here.

PAPL: the planner changes the training weight

Toy state A [MASK] G [MASK], with target C and T at the masks. The denoiser gives them probabilities 0.5 and 0.25. Compare equal weights with planner weights 0.8 and 0.2 at a fixed time.

$$\ell_{\text{equal}} = \tfrac{1}{2}[-\log(0.5)] + \tfrac{1}{2}[-\log(0.25)] \approx 1.040$$

With normalized equal weights, the two masked positions contribute equally to the average loss.

$$\ell_{\text{planner}} = 0.8[-\log(0.5)] + 0.2[-\log(0.25)] \approx 0.832$$

The planner puts four times as much training weight on the C position. A common time factor is omitted here.

Training emphasizes the positions that this planner is likely to reveal next.

We reweight the rate loss by the planner's choice of path

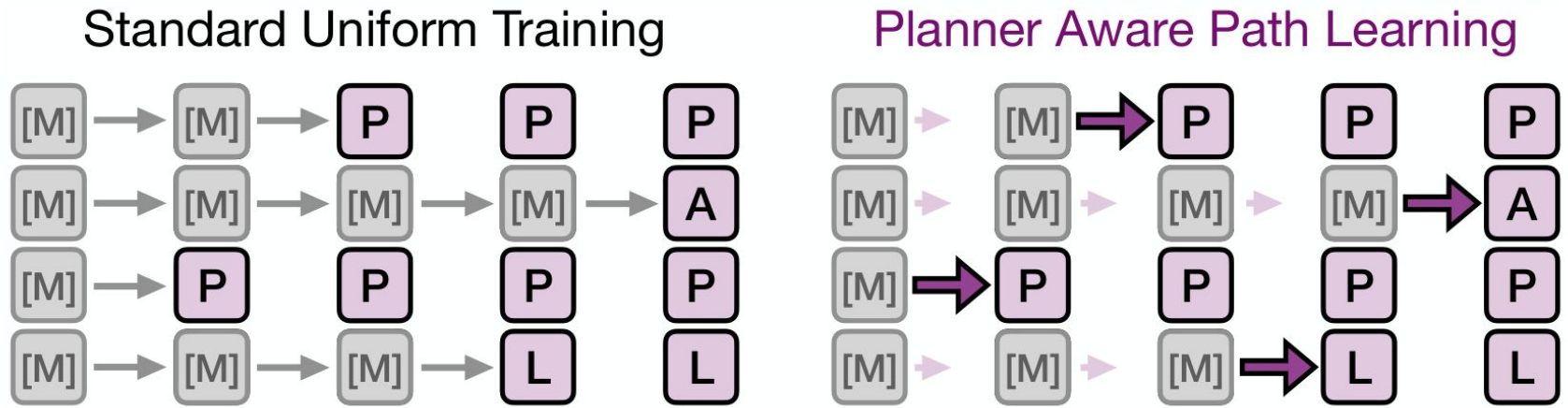


Figure 1: **Planner-Aware Path Learning (PAPL) resolves training–inference mismatch in DLMs.** Standard uniform training for DLMs (left) applies a uniform loss across all masked positions, distributing capacity over regions that inference-time planners never traverse. PAPL (right) introduces planner-aware weights into the loss, aligning training with the planner's preferred trajectories (outlined arrows) and eliminating training-inference mismatch.

Peng,..,Bose. *ICLR* (2026)

Here is the protein benchmark again, in Table 1 of the paper

Table 1: Protein sequence generation benchmark. We evaluate structure quality via pLDDT, pTM, and pAE, and diversity via token entropy and sequence uniqueness. Foldability is the percentage of sequences satisfying pLDDT > 80, pTM > 0.7, and pAE < 10.

	Model	pLDDT↑	pTM↑	pAE↓	Foldability (%)↑	Entropy↑	Diversity (%)↑
Large	ESM3	34.13	0.23	24.65	1.50	3.99	**93.44**
	ProGen2-medium	57.94	0.38	20.81	12.75	2.91	91.45
	ProGen2-large	55.07	0.35	22.00	11.87	2.73	91.48
	DPLM-650M	79.53	0.66	11.85	49.14	3.18	92.22
150M-scale	EvoDiff	31.84	0.21	24.76	0.43	**4.05**	93.19
	ProGen2-small	49.38	0.28	23.38	4.48	2.55	89.31
	DPLM-150M	80.23	0.65	12.07	48.14	3.14	92.80
	DLM-150M	81.32	0.65	12.00	42.43	3.21	92.45
	DLM-150M + PAPL (ours)	**81.48**	**0.72**	**8.97**	**59.40**	3.12	91.73

Peng,..,Bose. *ICLR* (2026)

The same 150M diffusion model folds 42.4% of its samples

Table 1: Protein sequence generation benchmark. We evaluate structure quality via pLDDT, pTM, and pAE, and diversity via token entropy and sequence uniqueness. Foldability is the percentage of sequences satisfying pLDDT > 80, pTM > 0.7, and pAE < 10.

	Model	pLDDT↑	pTM↑	pAE↓	Foldability (%)↑	Entropy↑	Diversity (%)↑
Large	ESM3	34.13	0.23	24.65	1.50	3.99	**93.44**
	ProGen2-medium	57.94	0.38	20.81	12.75	2.91	91.45
	ProGen2-large	55.07	0.35	22.00	11.87	2.73	91.48
	DPLM-650M	79.53	0.66	11.85	49.14	3.18	92.22
150M-scale	EvoDiff	31.84	0.21	24.76	0.43	**4.05**	93.19
	ProGen2-small	49.38	0.28	23.38	4.48	2.55	89.31
	DPLM-150M	80.23	0.65	12.07	48.14	3.14	92.80
	DLM-150M	81.32	0.65	12.00	42.43	3.21	92.45
	DLM-150M + PAPL (ours)	**81.48**	**0.72**	**8.97**	**59.40**	3.12	91.73

Peng,..,Bose. *ICLR* (2026)

With planner-aware training, that rises to 59.4%, and pAE falls from 12.00 to 8.97

Table 1: Protein sequence generation benchmark. We evaluate structure quality via pLDDT, pTM, and pAE, and diversity via token entropy and sequence uniqueness. Foldability is the percentage of sequences satisfying pLDDT > 80, pTM > 0.7, and pAE < 10.

	Model	pLDDT↑	pTM↑	pAE↓	Foldability (%)↑	Entropy↑	Diversity (%)↑
Large	ESM3	34.13	0.23	24.65	1.50	3.99	**93.44**
	ProGen2-medium	57.94	0.38	20.81	12.75	2.91	91.45
	ProGen2-large	55.07	0.35	22.00	11.87	2.73	91.48
	DPLM-650M	79.53	0.66	11.85	49.14	3.18	92.22
150M-scale	EvoDiff	31.84	0.21	24.76	0.43	**4.05**	93.19
	ProGen2-small	49.38	0.28	23.38	4.48	2.55	89.31
	DPLM-150M	80.23	0.65	12.07	48.14	3.14	92.80
	DLM-150M	81.32	0.65	12.00	42.43	3.21	92.45
	DLM-150M + PAPL (ours)	**81.48**	**0.72**	**8.97**	**59.40**	3.12	91.73

Peng,..,Bose. *ICLR* (2026)

PAPL is a reweighted rate loss, with the same rates and the same bound

Table 1: Protein sequence generation benchmark. We evaluate structure quality via pLDDT, pTM, and pAE, and diversity via token entropy and sequence uniqueness. Foldability is the percentage of sequences satisfying pLDDT > 80, pTM > 0.7, and pAE < 10.

	Model	pLDDT↑	pTM↑	pAE↓	Foldability (%)↑	Entropy↑	Diversity (%)↑
Large	ESM3	34.13	0.23	24.65	1.50	3.99	**93.44**
	ProGen2-medium	57.94	0.38	20.81	12.75	2.91	91.45
	ProGen2-large	55.07	0.35	22.00	11.87	2.73	91.48
	DPLM-650M	79.53	0.66	11.85	49.14	3.18	92.22
150M-scale	EvoDiff	31.84	0.21	24.76	0.43	**4.05**	93.19
	ProGen2-small	49.38	0.28	23.38	4.48	2.55	89.31
	DPLM-150M	80.23	0.65	12.07	48.14	3.14	92.80
	DLM-150M	81.32	0.65	12.00	42.43	3.21	92.45
	DLM-150M + PAPL (ours)	**81.48**	**0.72**	**8.97**	**59.40**	3.12	91.73

Peng,..,Bose. *ICLR* (2026)

How can we steer these samples toward peptides that bind, dissolve, and stay stable?

PepTune

PepTune: *De Novo* Generation of Therapeutic Peptides with Multi-Objective-Guided Discrete Diffusion

Sophia Tang [*1] **Yinuo Zhang** [*2] **Pranam Chatterjee** [1 3]

Tang,..,Chatterjee. *ICML* (2025)

Now generate sequences with the properties we need

Now generate sequences with the properties we need

So far, we have modeled the data distribution. For therapeutic design, we also need peptides to bind, stay soluble, and cross a membrane.

Now generate sequences with the properties we need

So far, we have modeled the data distribution. For therapeutic design, we also need peptides to bind, stay soluble, and cross a membrane.

Recall from Doob’s h-transform how to condition a jump process on a property, with $h_t(x) = p(c \mid X_t = x)$.

Now generate sequences with the properties we need

So far, we have modeled the data distribution. For therapeutic design, we also need peptides to bind, stay soluble, and cross a membrane.

Recall from Doob's h-transform how to condition a jump process on a property, with $h_t(x) = p(c \mid X_t = x)$.

In PepTune, we estimate that value by rolling out the denoiser to complete sequences and scoring them.

PepMDLM is masked diffusion over peptide SMILES, with every token a chemical fragment

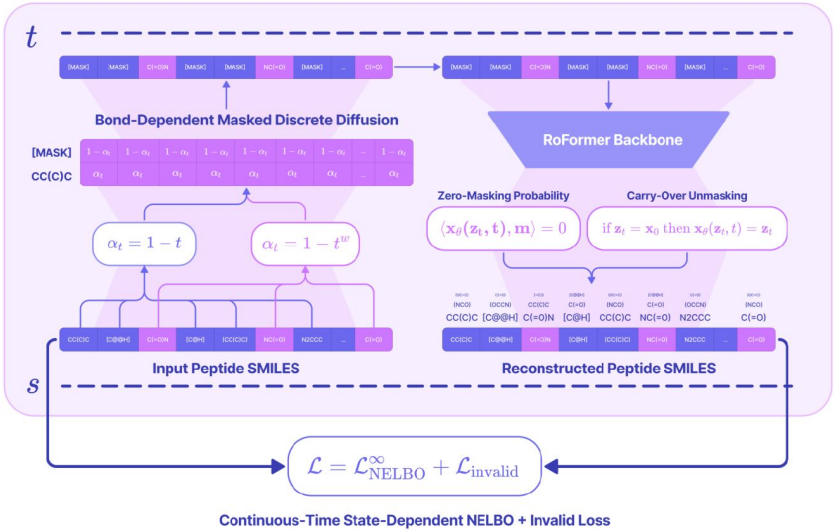


Figure 1. **PepMDLM**. PepMDLM is a discrete masked diffusion model for unconditional *de novo* generation of peptide SMILES representations.

Tang,..,Chatterjee. *ICML* (2025)

Next, we estimate the value h with Monte Carlo tree search

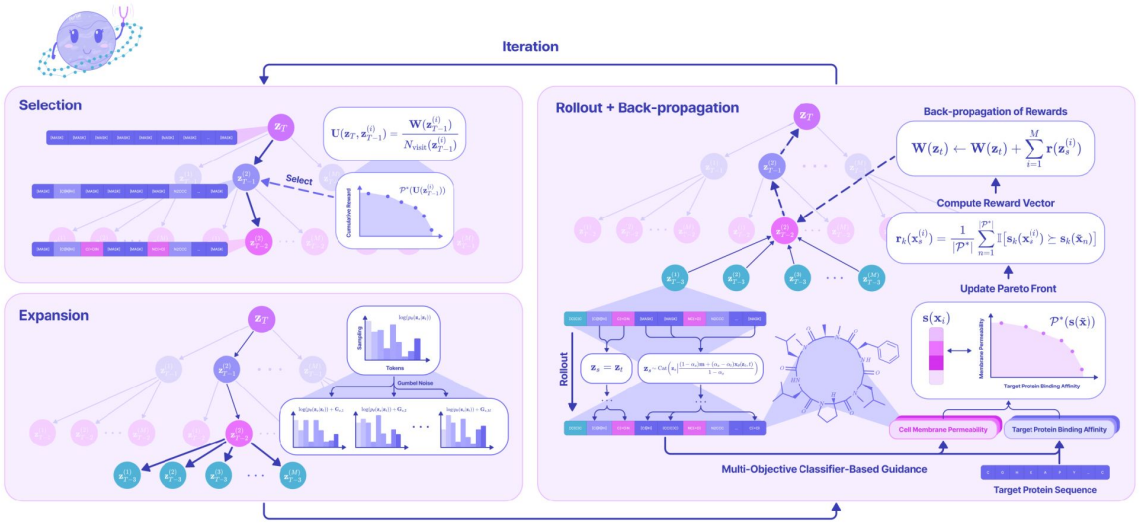


Figure 2. **PepTune**. PepTune is a multi-objective discrete diffusion model with Monte Carlo Tree Guidance (MCTG). The full algorithm is detailed in Algorithm 3.

Tang,..,Chatterjee. *ICML* (2025)

In selection, we follow a branch, balancing reward against how often it was explored

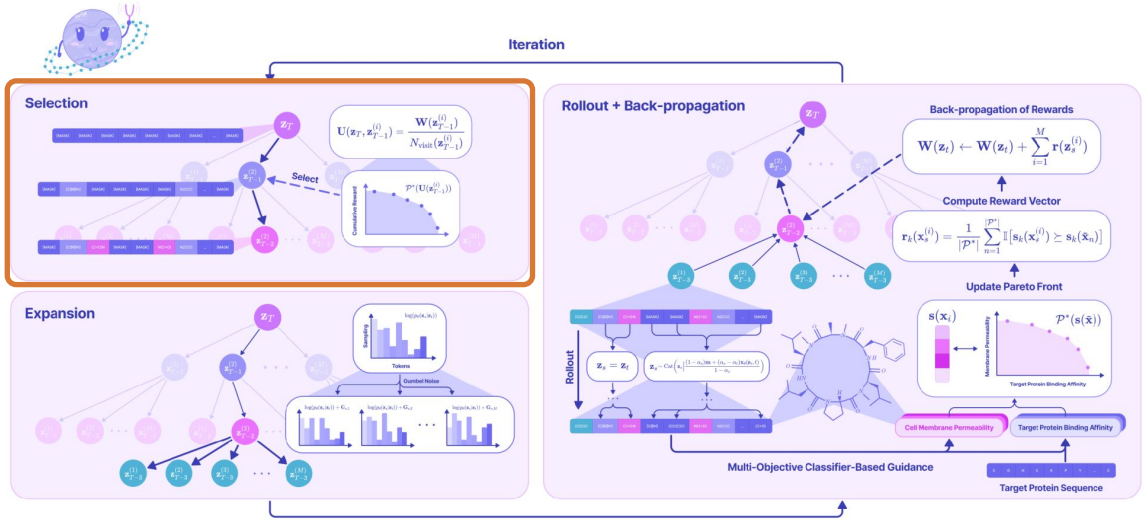


Figure 2. **PepTune**. PepTune is a multi-objective discrete diffusion model with Monte Carlo Tree Guidance (MCTG). The full algorithm is detailed in Algorithm 3.

Tang,..,Chatterjee. *ICML* (2025)

In expansion, the next unmasking comes from the denoiser, with Gumbel noise

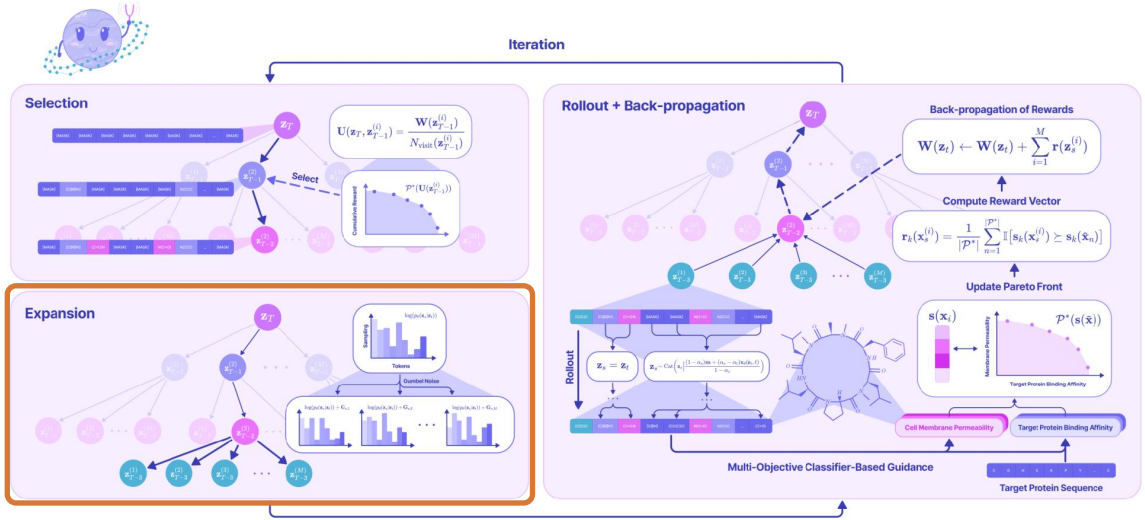


Figure 2. **PepTune**. PepTune is a multi-objective discrete diffusion model with Monte Carlo Tree Guidance (MCTG). The full algorithm is detailed in Algorithm 3.

Tang,..,Chatterjee. *ICML* (2025)

In each rollout, we finish the peptide and score it on every objective

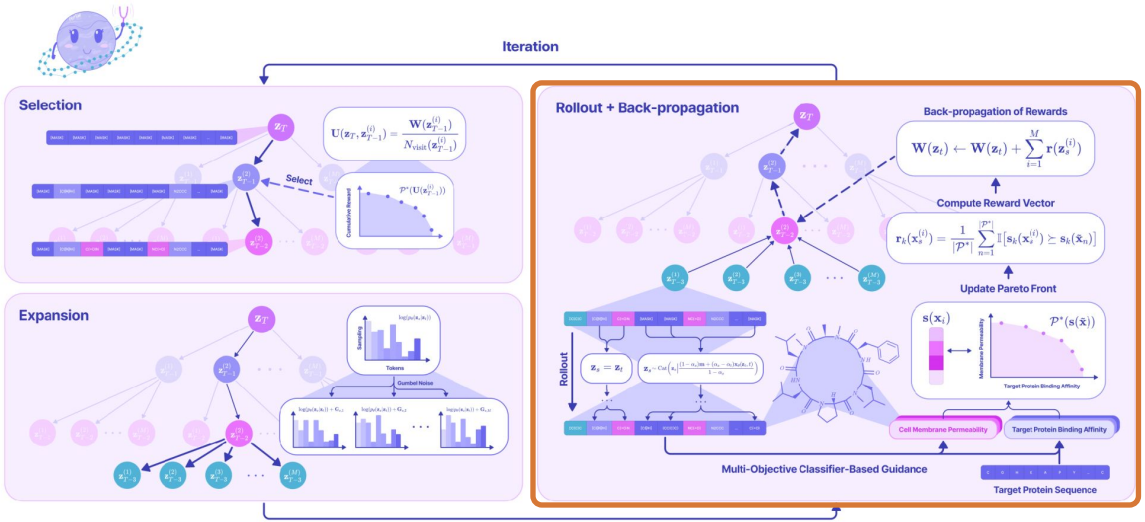


Figure 2. **PepTune**. PepTune is a multi-objective discrete diffusion model with Monte Carlo Tree Guidance (MCTG). The full algorithm is detailed in Algorithm 3.

Tang,..,Chatterjee. *ICML* (2025)

Those scores estimate h_t, and the rewards flow back up the tree

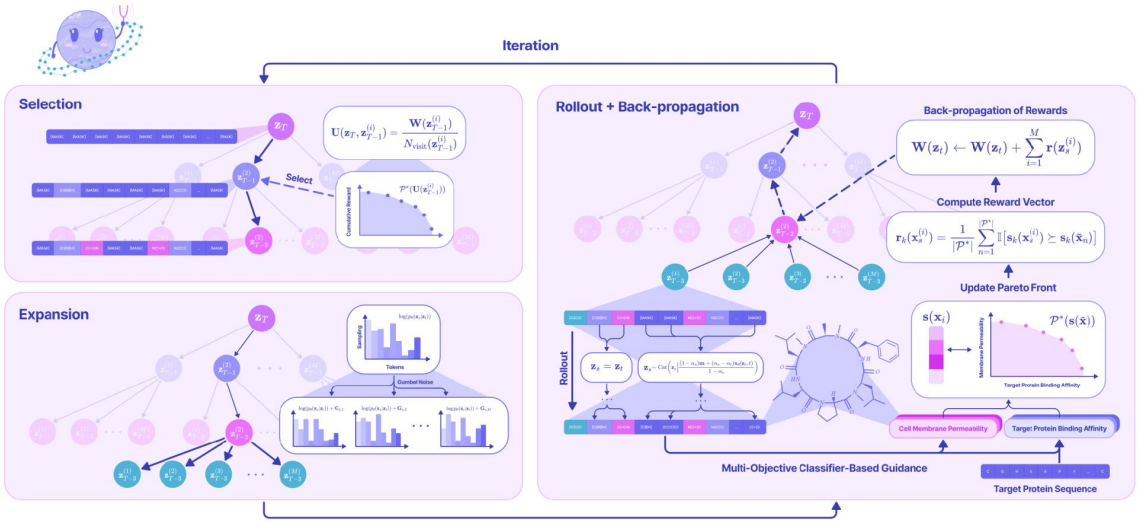


Figure 2. **PepTune**. PepTune is a multi-objective discrete diffusion model with Monte Carlo Tree Guidance (MCTG). The full algorithm is detailed in Algorithm 3.

Tang,..,Chatterjee. *ICML* (2025)

Now write the rollout estimate of the value

Now write the rollout estimate of the value

In each rollout, we complete z and score it, so the average reward estimates the value:

$$h_t(z) \approx \frac{1}{N} \sum_{n=1}^{N} r\left(x_0^{(n)}\right), \qquad x_0^{(n)} \sim p_\theta(\cdot \mid z)$$

Now write the rollout estimate of the value

In each rollout, we complete z and score it, so the average reward estimates the value:

$$h_t(z) \approx \frac{1}{N} \sum_{n=1}^{N} r\left(x_0^{(n)}\right), \qquad x_0^{(n)} \sim p_\theta(\cdot \mid z)$$

The value $h_t(z)$ is approximately the average over N rollouts of the reward $r(x_0^{(n)})$, where each completed sequence $x_0^{(n)}$ is sampled from the model $p_\theta(\cdot \mid z)$ starting from z.

Now write the rollout estimate of the value

In each rollout, we complete z and score it, so the average reward estimates the value:

$$h_t(z) \approx \frac{1}{N} \sum_{n=1}^{N} r\big(x_0^{(n)}\big), \qquad x_0^{(n)} \sim p_\theta(\cdot \mid z)$$

Then apply the h-transform to each reverse jump: $\hat{R}_t^c(z, z') = \hat{R}_t(z, z') \, \dfrac{h_t(z')}{h_t(z)}$

Now write the rollout estimate of the value

In each rollout, we complete z and score it, so the average reward estimates the value:

$$h_t(z) \approx \frac{1}{N} \sum_{n=1}^{N} r\big(x_0^{(n)}\big), \qquad x_0^{(n)} \sim p_\theta(\cdot \mid z)$$

Then apply the h-transform to each reverse jump: $\hat{R}_t^c(z, z') = \hat{R}_t(z, z') \, \frac{h_t(z')}{h_t(z)}$

The guided reverse rate $\hat{R}^c_t(z, z')$ from z to z′ equals the model's reverse rate $\hat{R}_t(z, z')$ times the estimated value $h_t(z')$ of z′ divided by the value $h_t(z)$ of z.

Now write the rollout estimate of the value

In each rollout, we complete z and score it, so the average reward estimates the value:

$$h_t(z) \approx \frac{1}{N} \sum_{n=1}^{N} r\big(x_0^{(n)}\big), \qquad x_0^{(n)} \sim p_\theta(\cdot \mid z)$$

Then apply the h-transform to each reverse jump: $\hat{R}_t^c(z, z') = \hat{R}_t(z, z') \, \dfrac{h_t(z')}{h_t(z)}$

With several objectives, no single reward orders the candidates, so in PepTune we keep every nondominated sequence in a Pareto archive.

Now write the rollout estimate of the value

In each rollout, we complete z and score it, so the average reward estimates the value:

$$h_t(z) \approx \frac{1}{N} \sum_{n=1}^{N} r\big(x_0^{(n)}\big), \qquad x_0^{(n)} \sim p_\theta(\cdot \mid z)$$

Then apply the h-transform to each reverse jump: $\hat{R}_t^c(z, z') = \hat{R}_t(z, z') \, \dfrac{h_t(z')}{h_t(z)}$

With several objectives, no single reward orders the candidates, so in PepTune we keep every nondominated sequence in a Pareto archive.

In selection, we balance reward against how often a branch has been explored, which is the usual tree-search trade-off.

Now write the rollout estimate of the value

In each rollout, we complete z and score it, so the average reward estimates the value:

$$h_t(z) \approx \frac{1}{N} \sum_{n=1}^{N} r\big(x_0^{(n)}\big), \qquad x_0^{(n)} \sim p_\theta(\cdot \mid z)$$

Then apply the h-transform to each reverse jump: $\hat{R}_t^c(z, z') = \hat{R}_t(z, z') \, \dfrac{h_t(z')}{h_t(z)}$

With several objectives, no single reward orders the candidates, so in PepTune we keep every nondominated sequence in a Pareto archive.

A partial peptide gets a high value when its sampled completions tend to have the properties we want.

PepTune: two rollouts change a jump rate

Use a toy scalar reward for two candidate peptide edits P and Q. Their rollout rewards are (0.2, 0.8) and (0.4, 0.4). Base rates are 2 and 1, and the current estimated value is 0.45.

PepTune: two rollouts change a jump rate

Use a toy scalar reward for two candidate peptide edits P and Q. Their rollout rewards are (0.2, 0.8) and (0.4, 0.4). Base rates are 2 and 1, and the current estimated value is 0.45.

$$\hat{h}(P) = \frac{0.2 + 0.8}{2} = 0.5, \qquad \hat{h}(Q) = \frac{0.4 + 0.4}{2} = 0.4$$

PepTune: two rollouts change a jump rate

Use a toy scalar reward for two candidate peptide edits P and Q. Their rollout rewards are (0.2, 0.8) and (0.4, 0.4). Base rates are 2 and 1, and the current estimated value is 0.45.

$$\hat{h}(P) = \frac{0.2 + 0.8}{2} = 0.5, \qquad \hat{h}(Q) = \frac{0.4 + 0.4}{2} = 0.4$$

Each candidate gets the mean reward of its completed rollouts.

PepTune: two rollouts change a jump rate

Use a toy scalar reward for two candidate peptide edits P and Q. Their rollout rewards are (0.2, 0.8) and (0.4, 0.4). Base rates are 2 and 1, and the current estimated value is 0.45.

$$\hat{h}(P) = \frac{0.2 + 0.8}{2} = 0.5, \qquad \hat{h}(Q) = \frac{0.4 + 0.4}{2} = 0.4$$

Each candidate gets the mean reward of its completed rollouts.

$$R^c(P) = 2\frac{0.5}{0.45} \approx 2.222, \qquad R^c(Q) = 1\frac{0.4}{0.45} \approx 0.889$$

PepTune: two rollouts change a jump rate

Use a toy scalar reward for two candidate peptide edits P and Q. Their rollout rewards are (0.2, 0.8) and (0.4, 0.4). Base rates are 2 and 1, and the current estimated value is 0.45.

$$\hat{h}(P) = \frac{0.2 + 0.8}{2} = 0.5, \qquad \hat{h}(Q) = \frac{0.4 + 0.4}{2} = 0.4$$

Each candidate gets the mean reward of its completed rollouts.

$$R^c(P) = 2\frac{0.5}{0.45} \approx 2.222, \qquad R^c(Q) = 1\frac{0.4}{0.45} \approx 0.889$$

Conditioned on a jump to one of these candidates, P now receives about 71.4% of the probability.

PepTune: two rollouts change a jump rate

Use a toy scalar reward for two candidate peptide edits P and Q. Their rollout rewards are (0.2, 0.8) and (0.4, 0.4). Base rates are 2 and 1, and the current estimated value is 0.45.

$$\hat{h}(P) = \frac{0.2 + 0.8}{2} = 0.5, \qquad \hat{h}(Q) = \frac{0.4 + 0.4}{2} = 0.4$$

Each candidate gets the mean reward of its completed rollouts.

$$R^c(P) = 2\frac{0.5}{0.45} \approx 2.222, \qquad R^c(Q) = 1\frac{0.4}{0.45} \approx 0.889$$

Conditioned on a jump to one of these candidates, P now receives about 71.4% of the probability.

A partially generated peptide gets more sampling effort when its continuations look promising.

Here is the peptide quality in Table 1 of the paper

Table 1. **Evaluation metrics for generative quality of peptide SMILES sequences of max token length set to 200.**

Model	Validity (↑)	Uniqueness (↑)	Diversity (↑)	SNN (↓)	Randomness (↑)	KL-Divergence (↑)
Data	1.000	1.000	0.885	1.000	4.55	0 (Reference)
PepMDLM	0.450	1.000	0.705	0.513	4.11	0.174
PepTune	1.000	1.000	0.677	0.486	4.12	0.173

Tang,..,Chatterjee. *ICML* (2025)

The unguided peptide model produces valid SMILES 45% of the time

Table 1. **Evaluation metrics for generative quality of peptide SMILES sequences of max token length set to 200.**

Model	Validity (↑)	Uniqueness (↑)	Diversity (↑)	SNN (↓)	Randomness (↑)	KL-Divergence (↑)
Data	1.000	1.000	0.885	1.000	4.55	0 (Reference)
PepMDLM	0.450	1.000	0.705	0.513	4.11	0.174
PepTune	1.000	1.000	0.677	0.486	4.12	0.173

Tang,..,Chatterjee. *ICML* (2025)

Searching under the h-transform reaches 100% validity at the same diversity

Table 1. **Evaluation metrics for generative quality of peptide SMILES sequences of max token length set to 200.**

Model	Validity (↑)	Uniqueness (↑)	Diversity (↑)	SNN (↓)	Randomness (↑)	KL-Divergence (↑)
Data	1.000	1.000	0.885	1.000	4.55	0 (Reference)
PepMDLM	0.450	1.000	0.705	0.513	4.11	0.174
PepTune	1.000	1.000	0.677	0.486	4.12	0.173

Tang,..,Chatterjee. *ICML* (2025)

The model is unchanged, and only its reverse rates are reweighted

Table 1. **Evaluation metrics for generative quality of peptide SMILES sequences of max token length set to 200.**

Model	Validity (↑)	Uniqueness (↑)	Diversity (↑)	SNN (↓)	Randomness (↑)	KL-Divergence (↑)
Data	1.000	1.000	0.885	1.000	4.55	0 (Reference)
PepMDLM	0.450	1.000	0.705	0.513	4.11	0.174
PepTune	1.000	1.000	0.677	0.486	4.12	0.173

Tang,..,Chatterjee. *ICML* (2025)

If search finds better generation paths, could the model learn to follow them on its own?

TR2-D2

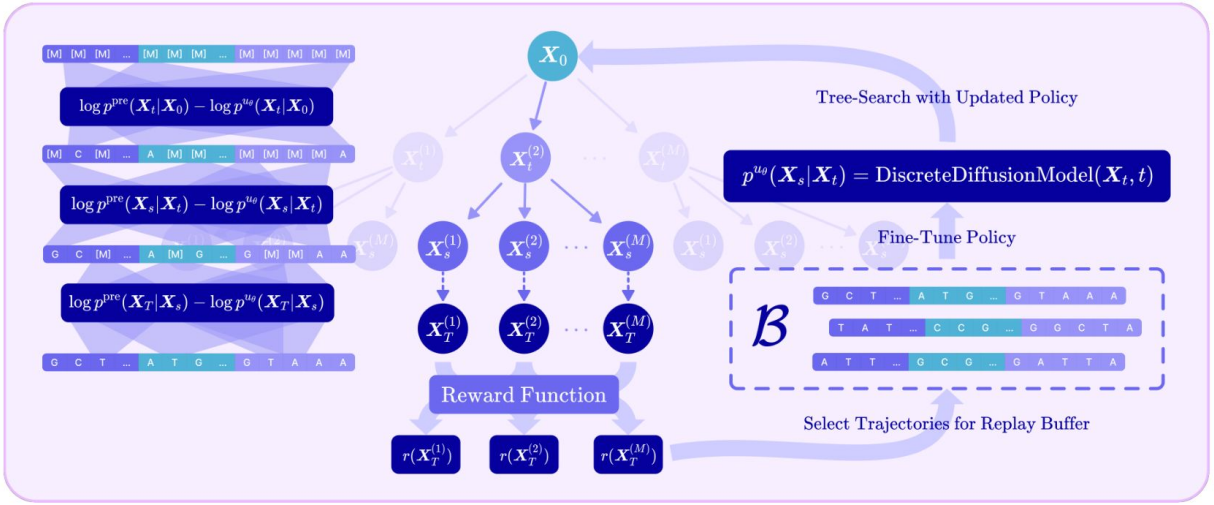


Figure 1: **Tree Search Guided Trajectory-Aware Fine-Tuning for Discrete Diffusion.** Our framework has two key components: **(1) a tree search algorithm** to generate a replay buffer of diffusion trajectories optimized for one or more reward functions using the current policy and **(2) an off-policy RL algorithm** for discrete diffusion fine-tuning using the optimized replay buffer.

Tang,..,Chatterjee. *arXiv* (2025)

Next, we want to keep the results of the search

Next, we want to keep the results of the search

In PepTune, we search at generation time, so we repeat the rollouts for every sample.

Next, we want to keep the results of the search

In PepTune, we search at generation time, so we repeat the rollouts for every sample.

In TR2-D2, we keep the trajectories from the search and fine-tune the model on them.

Next, we want to keep the results of the search

In PepTune, we search at generation time, so we repeat the rollouts for every sample.

In TR2-D2, we keep the trajectories from the search and fine-tune the model on them.

The target is the reward-tilted distribution: $$\boxed{p^*(x) \propto p_\theta(x)\ \exp\left(r(x)/\lambda\right)}$$

Next, we want to keep the results of the search

In PepTune, we search at generation time, so we repeat the rollouts for every sample.

In TR2-D2, we keep the trajectories from the search and fine-tune the model on them.

The target is the reward-tilted distribution: $$p^*(x) \propto p_\theta(x)\ \exp\left(r(x)/\lambda\right)$$

The target distribution $p^*(x)$ is proportional to the model probability $p_\theta(x)$ times the exponential of the reward $r(x)$ divided by the temperature λ.

Next, we want to keep the results of the search

In PepTune, we search at generation time, so we repeat the rollouts for every sample.

In TR2-D2, we keep the trajectories from the search and fine-tune the model on them.

The target is the reward-tilted distribution: $$p^*(x) \propto p_\theta(x)\ \exp\left(r(x)/\lambda\right)$$

The target distribution $p^*(x)$ is proportional to the model probability $p_\theta(x)$ times the exponential of the reward $r(x)$ divided by the temperature λ.

This is the h-transform again, with $h(x) = \exp(r(x)/\lambda)$ at the end of generation.

Next, we want to keep the results of the search

In PepTune, we search at generation time, so we repeat the rollouts for every sample.

In TR2-D2, we keep the trajectories from the search and fine-tune the model on them.

The target is the reward-tilted distribution: $$p^*(x) \propto p_\theta(x)\ \exp\left(r(x)/\lambda\right)$$

The target distribution $p^*(x)$ is proportional to the model probability $p_\theta(x)$ times the exponential of the reward $r(x)$ divided by the temperature λ.

Lower temperature makes a fixed reward difference change the target probabilities more strongly.

We explore reverse paths of the current model by tree search

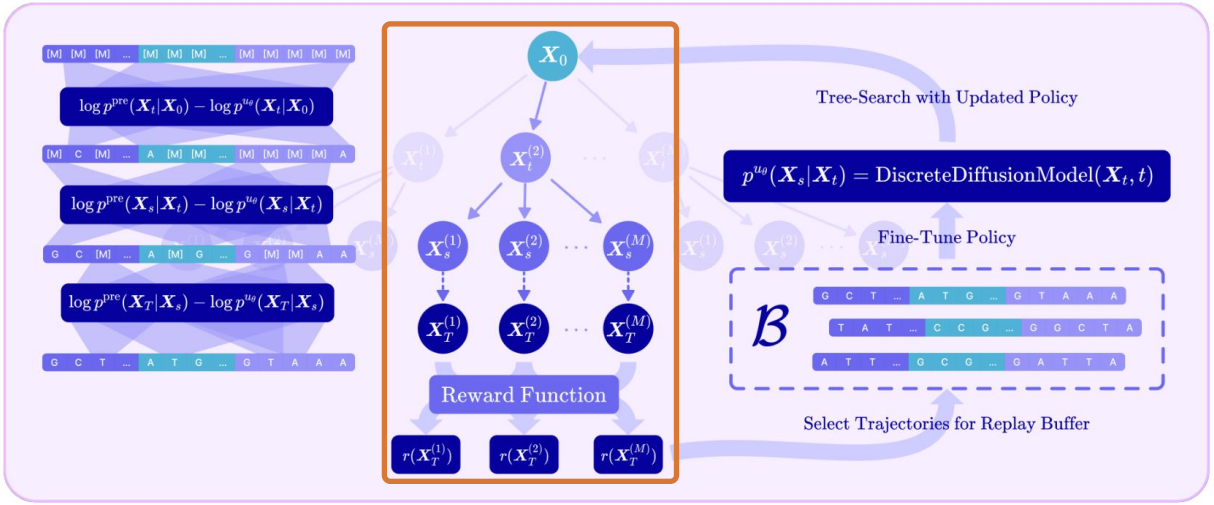


Figure 1: **Tree Search Guided Trajectory-Aware Fine-Tuning for Discrete Diffusion.** Our framework has two key components: **(1) a tree search algorithm** to generate a replay buffer of diffusion trajectories optimized for one or more reward functions using the current policy and **(2) an off-policy RL algorithm** for discrete diffusion fine-tuning using the optimized replay buffer.

Tang,..,Chatterjee. *arXiv* (2025)

Each completed path has a weight, the reward against its path probability

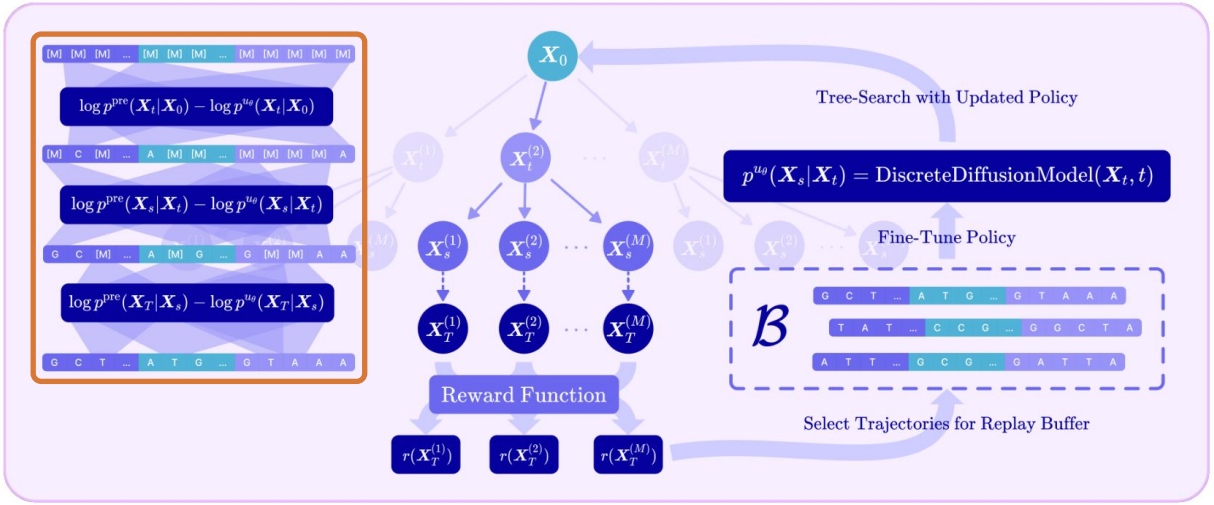


Figure 1: **Tree Search Guided Trajectory-Aware Fine-Tuning for Discrete Diffusion.** Our framework has two key components: **(1) a tree search algorithm** to generate a replay buffer of diffusion trajectories optimized for one or more reward functions using the current policy and **(2) an off-policy RL algorithm** for discrete diffusion fine-tuning using the optimized replay buffer.

Tang,..,Chatterjee. *arXiv* (2025)

When we train on those weighted paths, the model moves toward the tilted target

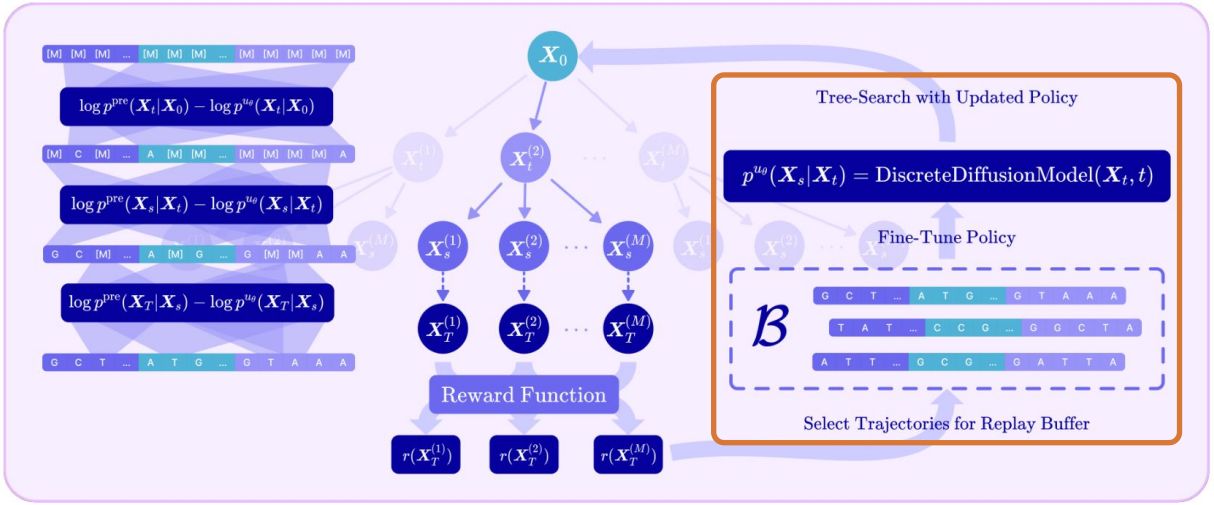


Figure 1: **Tree Search Guided Trajectory-Aware Fine-Tuning for Discrete Diffusion.** Our framework has two key components: **(1) a tree search algorithm** to generate a replay buffer of diffusion trajectories optimized for one or more reward functions using the current policy and **(2) an off-policy RL algorithm** for discrete diffusion fine-tuning using the optimized replay buffer.

Tang,..,Chatterjee. *arXiv* (2025)

The weights are products of the rate ratios we have been computing

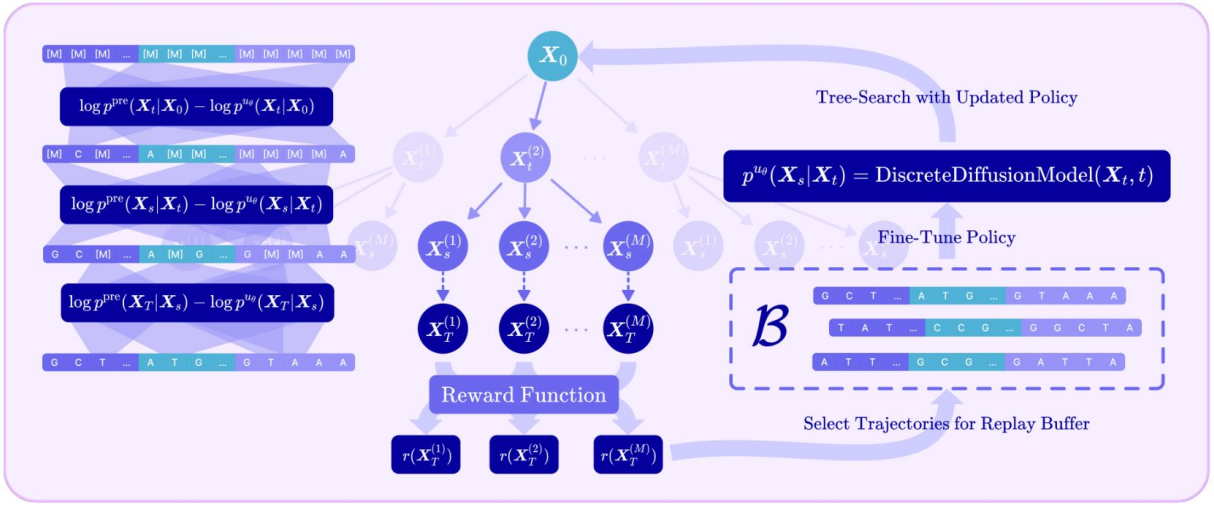


Figure 1: **Tree Search Guided Trajectory-Aware Fine-Tuning for Discrete Diffusion.** Our framework has two key components: **(1) a tree search algorithm** to generate a replay buffer of diffusion trajectories optimized for one or more reward functions using the current policy and **(2) an off-policy RL algorithm** for discrete diffusion fine-tuning using the optimized replay buffer.

Tang,..,Chatterjee. *arXiv* (2025)

Next, we write the weight of a path

Next, we write the weight of a path

Two rollouts can reach the same sequence along different paths, and the one that followed the model more closely should get more weight.

Next, we write the weight of a path

Two rollouts can reach the same sequence along different paths, and the one that followed the model more closely should get more weight.

So in the weight, we compare the two sets of rates along the path:

$$\log W = \frac{r(x_0)}{\lambda} + \sum_{\text{jumps}} \log \frac{\hat{R}_t^{\theta}(z, z')}{\hat{R}_t^{\text{search}}(z, z')}$$

Next, we write the weight of a path

Two rollouts can reach the same sequence along different paths, and the one that followed the model more closely should get more weight.

So in the weight, we compare the two sets of rates along the path:

$$\log W = \frac{r(x_0)}{\lambda} + \sum_{\text{jumps}} \log \frac{\hat{R}_t^\theta(z, z')}{\hat{R}_t^{\text{search}}(z, z')}$$

The log weight log W of a path equals the reward $r(x_0)$ of its final sequence divided by the temperature λ, plus the sum over its jumps from z to z′ of the log of the model rate $\hat{R}^\theta_t(z, z')$ divided by the search rate $\hat{R}^{\text{search}}_t(z, z')$.

Next, we write the weight of a path

Two rollouts can reach the same sequence along different paths, and the one that followed the model more closely should get more weight.

So in the weight, we compare the two sets of rates along the path:

$$\log W = \frac{r(x_0)}{\lambda} + \sum_{\text{jumps}} \log \frac{\hat{R}_t^\theta(z, z')}{\hat{R}_t^{\text{search}}(z, z')}$$

The log weight log W of a path equals the reward $r(x_0)$ of its final sequence divided by the temperature λ, plus the sum over its jumps from z to z′ of the log of the model rate $\hat{R}^\theta_t(z, z')$ divided by the search rate $\hat{R}^{\text{search}}_t(z, z')$.

Training on these weighted trajectories is a trajectory-aware version of the rate loss.

Next, we write the weight of a path

Two rollouts can reach the same sequence along different paths, and the one that followed the model more closely should get more weight.

So in the weight, we compare the two sets of rates along the path:

$$\log W = \frac{r(x_0)}{\lambda} + \sum_{\text{jumps}} \log \frac{\hat{R}_t^\theta(z, z')}{\hat{R}_t^{\text{search}}(z, z')}$$

The log weight log W of a path equals the reward $r(x_0)$ of its final sequence divided by the temperature λ, plus the sum over its jumps from z to z′ of the log of the model rate $\hat{R}^\theta_t(z, z')$ divided by the search rate $\hat{R}^{\text{search}}_t(z, z')$.

Training on these weighted trajectories is a trajectory-aware version of the rate loss.

After fine-tuning, sampling costs no more than the base model.

TR2-D2: a reward changes the target distribution

Toy candidates ACGT and AGGT have base probabilities 0.8 and 0.2. Give them rewards 0 and log 4, with temperature 1.

TR2-D2: a reward changes the target distribution

Toy candidates ACGT and AGGT have base probabilities 0.8 and 0.2. Give them rewards 0 and log 4, with temperature 1.

$$(\widetilde{p}(\mathrm{ACGT}), \widetilde{p}(\mathrm{AGGT})) = (0.8e^{0},\ 0.2e^{\log 4}) = (0.8, 0.8)$$

TR2-D2: a reward changes the target distribution

Toy candidates ACGT and AGGT have base probabilities 0.8 and 0.2. Give them rewards 0 and log 4, with temperature 1.

$$(\widetilde{p}(\mathrm{ACGT}), \widetilde{p}(\mathrm{AGGT})) = (0.8e^{0},\ 0.2e^{\log 4}) = (0.8, 0.8)$$

The exponential reward multiplies the less common candidate by four.

TR2-D2: a reward changes the target distribution

Toy candidates ACGT and AGGT have base probabilities 0.8 and 0.2. Give them rewards 0 and log 4, with temperature 1.

$$(\widetilde{p}(\mathrm{ACGT}), \widetilde{p}(\mathrm{AGGT})) = (0.8e^{0},\ 0.2e^{\log 4}) = (0.8, 0.8)$$

The exponential reward multiplies the less common candidate by four.

$$p^{*}(\mathrm{ACGT}) = \frac{0.8}{1.6} = 0.5, \qquad p^{*}(\mathrm{AGGT}) = \frac{0.8}{1.6} = 0.5$$

TR2-D2: a reward changes the target distribution

Toy candidates ACGT and AGGT have base probabilities 0.8 and 0.2. Give them rewards 0 and log 4, with temperature 1.

$$(\widetilde{p}(\mathrm{ACGT}), \widetilde{p}(\mathrm{AGGT})) = (0.8e^{0},\ 0.2e^{\log 4}) = (0.8, 0.8)$$

The exponential reward multiplies the less common candidate by four.

$$p^{*}(\mathrm{ACGT}) = \frac{0.8}{1.6} = 0.5, \qquad p^{*}(\mathrm{AGGT}) = \frac{0.8}{1.6} = 0.5$$

Normalize the two weights. The rewarded candidate rises from 20% to 50% in the target law.

TR2-D2: a reward changes the target distribution

Toy candidates ACGT and AGGT have base probabilities 0.8 and 0.2. Give them rewards 0 and log 4, with temperature 1.

$$(\widetilde{p}(\mathrm{ACGT}), \widetilde{p}(\mathrm{AGGT})) = (0.8e^{0},\ 0.2e^{\log 4}) = (0.8, 0.8)$$

The exponential reward multiplies the less common candidate by four.

$$p^{*}(\mathrm{ACGT}) = \frac{0.8}{1.6} = 0.5, \qquad p^{*}(\mathrm{AGGT}) = \frac{0.8}{1.6} = 0.5$$

Normalize the two weights. The rewarded candidate rises from 20% to 50% in the target law.

Training on weighted search trajectories aims to make this preference available in the model itself.

Here is the regulatory DNA result in Table 1 of the paper

Table 1: **Comparison of TR2-D2 for regulatory DNA generation optimized on enhancer activity.** Metrics were computed for 640 sequences across 3 seeds, with standard deviations reported. Best values are **bolded**. Evaluation metrics are detailed in Appendix D.

Method	Pred-Activity (median; ↑)	ATAC-Acc (%; ↑)	3-mer Corr (↑)	App-Log-Lik (median; ↑)
pre-trained	$0.17_{\pm 0.04}$	$1.5_{\pm 0.2}$	$-0.061_{\pm 0.034}$	$-261_{\pm 0.6}$
CG	$3.30_{\pm 0.00}$	$0.0_{\pm 0.0}$	$-0.065_{\pm 0.001}$	$-266_{\pm 0.6}$
SMC	$4.15_{\pm 0.33}$	$39.9_{\pm 8.7}$	$0.840_{\pm 0.045}$	$-259_{\pm 2.5}$
TDS	$4.64_{\pm 0.21}$	$45.3_{\pm 16.4}$	$0.848_{\pm 0.008}$	$-257_{\pm 1.5}$
CFG	$5.04_{\pm 0.06}$	$92.1_{\pm 0.9}$	$0.746_{\pm 0.001}$	$-265_{\pm 0.6}$
DRAKES	$5.61_{\pm 0.07}$	$92.5_{\pm 0.6}$	$0.887_{\pm 0.002}$	$-264_{\pm 0.6}$
SEPO	$7.55_{\pm 0.01}$	$99.5_{\pm 0.2}$	$0.500_{\pm 0.004}$	$-243.8_{\pm 0.5}$
GLID^{2}E	$7.35_{\pm 0.07}$	$90.6_{\pm 0.3}$	$0.490_{\pm 0.074}$	$\mathbf{-239.9_{\pm 1.4}}$
TR2-D2 ($\alpha = 0.1$)	$6.56_{\pm 0.02}$	$86.9_{\pm 1.18}$	$\mathbf{0.925_{\pm 0.002}}$	$-259.4_{\pm 0.2}$
TR2-D2 ($\alpha = 0.001$)	$\mathbf{9.74_{\pm 0.01}}$	$\mathbf{99.9_{\pm 0.01}}$	$0.548_{\pm 0.001}$	$-271.8_{\pm 0.1}$

Tang,..,Chatterjee. *arXiv* (2025)

On enhancer DNA, the pre-trained model has median predicted activity 0.17

Table 1: **Comparison of TR2-D2 for regulatory DNA generation optimized on enhancer activity.** Metrics were computed for 640 sequences across 3 seeds, with standard deviations reported. Best values are **bolded**. Evaluation metrics are detailed in Appendix D.

Method	Pred-Activity (median; ↑)	ATAC-Acc (%; ↑)	3-mer Corr (↑)	App-Log-Lik (median; ↑)
pre-trained	$0.17_{\pm 0.04}$	$1.5_{\pm 0.2}$	$-0.061_{\pm 0.034}$	$-261_{\pm 0.6}$
CG	$3.30_{\pm 0.00}$	$0.0_{\pm 0.0}$	$-0.065_{\pm 0.001}$	$-266_{\pm 0.6}$
SMC	$4.15_{\pm 0.33}$	$39.9_{\pm 8.7}$	$0.840_{\pm 0.045}$	$-259_{\pm 2.5}$
TDS	$4.64_{\pm 0.21}$	$45.3_{\pm 16.4}$	$0.848_{\pm 0.008}$	$-257_{\pm 1.5}$
CFG	$5.04_{\pm 0.06}$	$92.1_{\pm 0.9}$	$0.746_{\pm 0.001}$	$-265_{\pm 0.6}$
DRAKES	$5.61_{\pm 0.07}$	$92.5_{\pm 0.6}$	$0.887_{\pm 0.002}$	$-264_{\pm 0.6}$
SEPO	$7.55_{\pm 0.01}$	$99.5_{\pm 0.2}$	$0.500_{\pm 0.004}$	$-243.8_{\pm 0.5}$
GLID^{2}E	$7.35_{\pm 0.07}$	$90.6_{\pm 0.3}$	$0.490_{\pm 0.074}$	$\mathbf{-239.9_{\pm 1.4}}$
TR2-D2 ($\alpha = 0.1$)	$6.56_{\pm 0.02}$	$86.9_{\pm 1.18}$	$\mathbf{0.925_{\pm 0.002}}$	$-259.4_{\pm 0.2}$
TR2-D2 ($\alpha = 0.001$)	$\mathbf{9.74_{\pm 0.01}}$	$\mathbf{99.9_{\pm 0.01}}$	$0.548_{\pm 0.001}$	$-271.8_{\pm 0.1}$

Tang,..,Chatterjee. *arXiv* (2025)

The fine-tuned model reaches 9.74, with 99.9% chromatin accessibility

Table 1: **Comparison of TR2-D2 for regulatory DNA generation optimized on enhancer activity.** Metrics were computed for 640 sequences across 3 seeds, with standard deviations reported. Best values are **bolded**. Evaluation metrics are detailed in Appendix D.

Method	Pred-Activity (median; ↑)	ATAC-Acc (%; ↑)	3-mer Corr (↑)	App-Log-Lik (median; ↑)
pre-trained	$0.17_{\pm 0.04}$	$1.5_{\pm 0.2}$	$-0.061_{\pm 0.034}$	$-261_{\pm 0.6}$
CG	$3.30_{\pm 0.00}$	$0.0_{\pm 0.0}$	$-0.065_{\pm 0.001}$	$-266_{\pm 0.6}$
SMC	$4.15_{\pm 0.33}$	$39.9_{\pm 8.7}$	$0.840_{\pm 0.045}$	$-259_{\pm 2.5}$
TDS	$4.64_{\pm 0.21}$	$45.3_{\pm 16.4}$	$0.848_{\pm 0.008}$	$-257_{\pm 1.5}$
CFG	$5.04_{\pm 0.06}$	$92.1_{\pm 0.9}$	$0.746_{\pm 0.001}$	$-265_{\pm 0.6}$
DRAKES	$5.61_{\pm 0.07}$	$92.5_{\pm 0.6}$	$0.887_{\pm 0.002}$	$-264_{\pm 0.6}$
SEPO	$7.55_{\pm 0.01}$	$99.5_{\pm 0.2}$	$0.500_{\pm 0.004}$	$-243.8_{\pm 0.5}$
GLID^{2}E	$7.35_{\pm 0.07}$	$90.6_{\pm 0.3}$	$0.490_{\pm 0.074}$	$\mathbf{-239.9_{\pm 1.4}}$
TR2-D2 ($\alpha = 0.1$)	$6.56_{\pm 0.02}$	$86.9_{\pm 1.18}$	$\mathbf{0.925_{\pm 0.002}}$	$-259.4_{\pm 0.2}$
TR2-D2 ($\alpha = 0.001$)	$\mathbf{9.74_{\pm 0.01}}$	$\mathbf{99.9_{\pm 0.01}}$	$0.548_{\pm 0.001}$	$-271.8_{\pm 0.1}$

Tang,..,Chatterjee. *arXiv* (2025)

Sampling then costs no more than the base model, with no rollouts

Table 1: **Comparison of TR2-D2 for regulatory DNA generation optimized on enhancer activity.** Metrics were computed for 640 sequences across 3 seeds, with standard deviations reported. Best values are **bolded**. Evaluation metrics are detailed in Appendix D.

Method	Pred-Activity (median; ↑)	ATAC-Acc (%; ↑)	3-mer Corr (↑)	App-Log-Lik (median; ↑)
pre-trained	$0.17_{\pm 0.04}$	$1.5_{\pm 0.2}$	$-0.061_{\pm 0.034}$	$-261_{\pm 0.6}$
CG	$3.30_{\pm 0.00}$	$0.0_{\pm 0.0}$	$-0.065_{\pm 0.001}$	$-266_{\pm 0.6}$
SMC	$4.15_{\pm 0.33}$	$39.9_{\pm 8.7}$	$0.840_{\pm 0.045}$	$-259_{\pm 2.5}$
TDS	$4.64_{\pm 0.21}$	$45.3_{\pm 16.4}$	$0.848_{\pm 0.008}$	$-257_{\pm 1.5}$
CFG	$5.04_{\pm 0.06}$	$92.1_{\pm 0.9}$	$0.746_{\pm 0.001}$	$-265_{\pm 0.6}$
DRAKES	$5.61_{\pm 0.07}$	$92.5_{\pm 0.6}$	$0.887_{\pm 0.002}$	$-264_{\pm 0.6}$
SEPO	$7.55_{\pm 0.01}$	$99.5_{\pm 0.2}$	$0.500_{\pm 0.004}$	$-243.8_{\pm 0.5}$
GLID^{2}E	$7.35_{\pm 0.07}$	$90.6_{\pm 0.3}$	$0.490_{\pm 0.074}$	$\mathbf{-239.9_{\pm 1.4}}$
TR2-D2 ($\alpha = 0.1$)	$6.56_{\pm 0.02}$	$86.9_{\pm 1.18}$	$\mathbf{0.925_{\pm 0.002}}$	$-259.4_{\pm 0.2}$
TR2-D2 ($\alpha = 0.001$)	$\mathbf{9.74_{\pm 0.01}}$	$\mathbf{99.9_{\pm 0.01}}$	$0.548_{\pm 0.001}$	$-271.8_{\pm 0.1}$

Tang,..,Chatterjee. *arXiv* (2025)

Could the model decide the sequence length while it generates the tokens?

A2D2

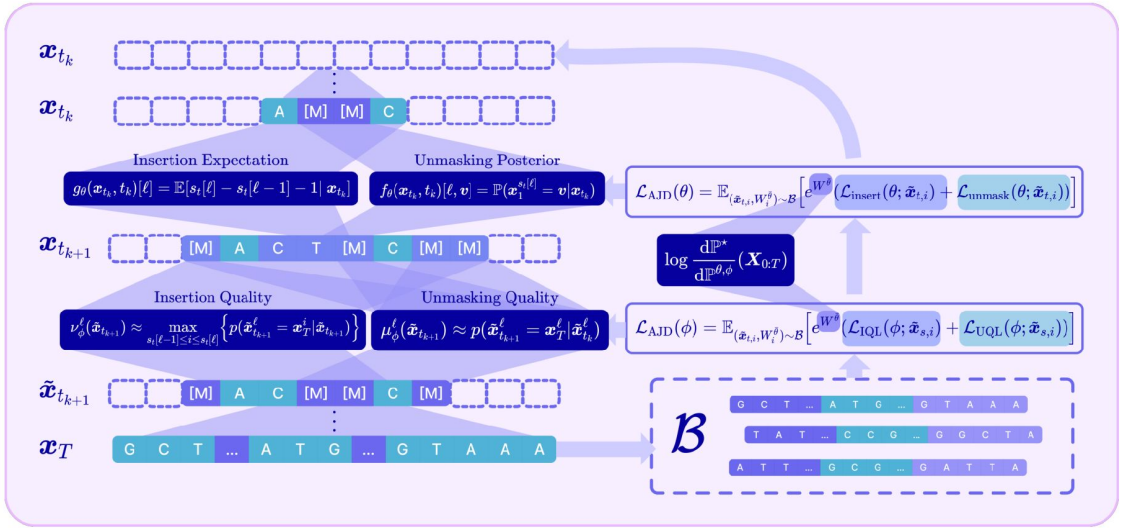


Figure 1 ***Fine-Tuning Any-Length Discrete Diffusion for Adaptive Decoding.*** *A2D2 jointly fine-tunes the insertion and unmasking policies together with a quality classifier to sample from the optimal reward-tilted distribution $\mathbb{P}^\star$. Sequences are generated via adaptive inference and stored in a replay buffer, then used to compute the Adaptive Joint Decoding loss $\mathcal{L}_{AJD}$, which weights each sequence's contribution by an importance weight $W^{\theta,\bar{\phi}}$.*

Tang,..,Chatterjee. *arXiv* (2026)

So far, we have fixed the length in advance

So far, we have fixed the length in advance

Our state space is sequences of length L. We also need to generate the length of a peptide or a sentence.

So far, we have fixed the length in advance

Our state space is sequences of length L. We also need to generate the length of a peptide or a sentence.

In A2D2, we add a second kind of jump, the insertion of a position, alongside unmasking.

So far, we have fixed the length in advance

Our state space is sequences of length L. We also need to generate the length of a peptide or a sentence.

In A2D2, we add a second kind of jump, the insertion of a position, alongside unmasking.

Both are transitions in one rate matrix, so the master equation, the reversal, and the rate loss carry over.

With an insertion policy, we choose how many positions to add at each gap

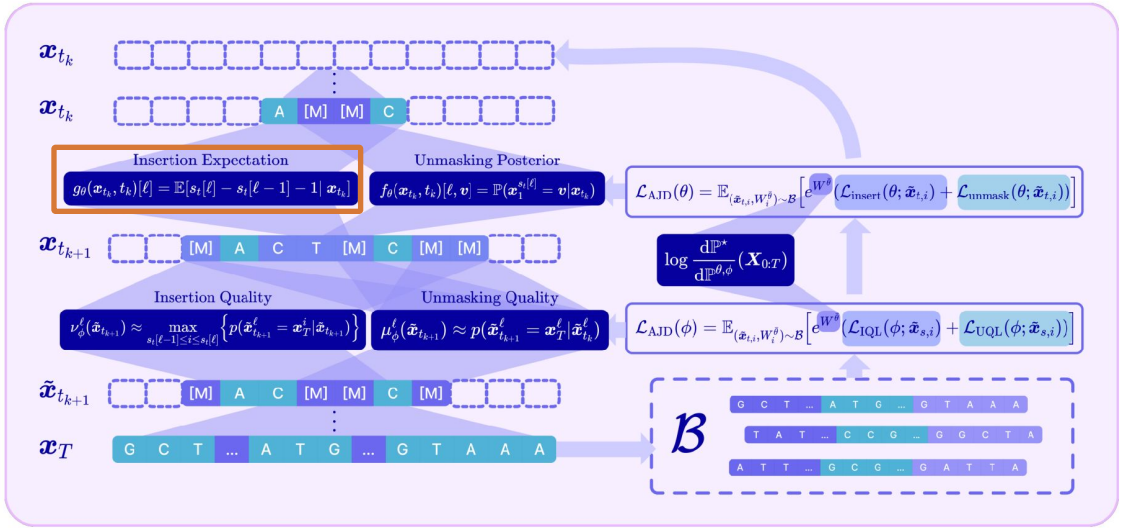


Figure 1 ***Fine-Tuning Any-Length Discrete Diffusion for Adaptive Decoding.*** *A2D2 jointly fine-tunes the insertion and unmasking policies together with a quality classifier to sample from the optimal reward-tilted distribution $\mathbb{P}^\star$. Sequences are generated via adaptive inference and stored in a replay buffer, then used to compute the Adaptive Joint Decoding loss $\mathcal{L}_{AJD}$, which weights each sequence's contribution by an importance weight $W^{\bar{\theta},\bar{\phi}}$.*

Tang,..,Chatterjee. *arXiv* (2026)

Then fill them with an unmasking policy, as in MDLM

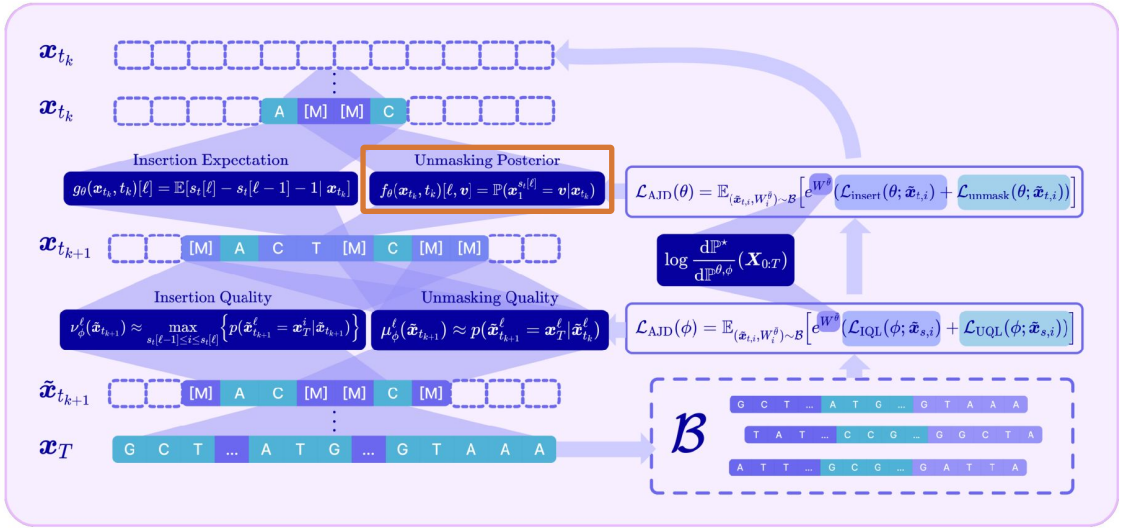


Figure 1 ***Fine-Tuning Any-Length Discrete Diffusion for Adaptive Decoding.*** *A2D2 jointly fine-tunes the insertion and unmasking policies together with a quality classifier to sample from the optimal reward-tilted distribution $\mathbb{P}^\star$. Sequences are generated via adaptive inference and stored in a replay buffer, then used to compute the Adaptive Joint Decoding loss $\mathcal{L}_{AJD}$, which weights each sequence's contribution by an importance weight $W^{\theta,\bar{\phi}}$.*

Tang,..,Chatterjee. *arXiv* (2026)

With a quality head, we score whether to trust each decision

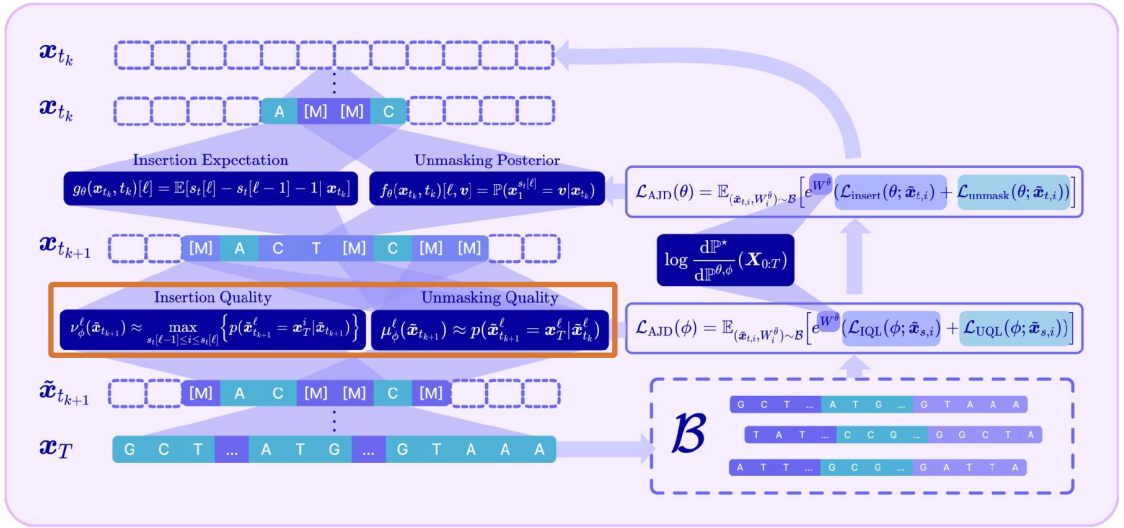


Figure 1 ***Fine-Tuning Any-Length Discrete Diffusion for Adaptive Decoding.*** *A2D2 jointly fine-tunes the insertion and unmasking policies together with a quality classifier to sample from the optimal reward-tilted distribution $\mathbb{P}^\star$. Sequences are generated via adaptive inference and stored in a replay buffer, then used to compute the Adaptive Joint Decoding loss $\mathcal{L}_{AJD}$, which weights each sequence's contribution by an importance weight $W^{\theta,\bar{\phi}}$.*

Tang,..,Chatterjee. *arXiv* (2026)

The sequence grows and fills at the same time

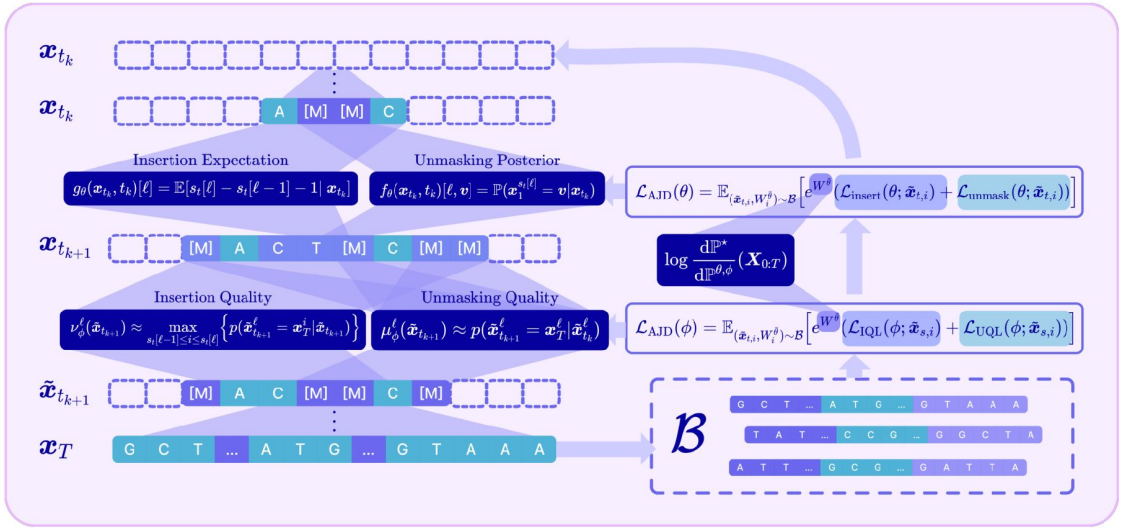


Figure 1 ***Fine-Tuning Any-Length Discrete Diffusion for Adaptive Decoding.*** *A2D2 jointly fine-tunes the insertion and unmasking policies together with a quality classifier to sample from the optimal reward-tilted distribution $\mathbb{P}^\star$. Sequences are generated via adaptive inference and stored in a replay buffer, then used to compute the Adaptive Joint Decoding loss $\mathcal{L}_{AJD}$, which weights each sequence's contribution by an importance weight $W^{\theta,\bar{\phi}}$.*

Tang,..,Chatterjee. *arXiv* (2026)

Now write the path weight when the length can change

Now write the path weight when the length can change

With insertions, the total rate of leaving a state depends on its length. We must account for the waiting time in the path likelihood.

Now write the path weight when the length can change

With insertions, the total rate of leaving a state depends on its length. We must account for the waiting time in the path likelihood.

The path KL we derived has these two terms:

$$\log W = \sum_{\text{jumps}} \log \frac{\hat{R}^{\theta}}{\hat{R}} - \int_0^1 \left(\hat{R}^{\theta}_{\text{out}} - \hat{R}_{\text{out}}\right) dt$$

Now write the path weight when the length can change

With insertions, the total rate of leaving a state depends on its length. We must account for the waiting time in the path likelihood.

The path KL we derived has these two terms:

$$\log W = \sum_{\text{jumps}} \log \frac{\hat{R}^{\theta}}{\hat{R}} - \int_0^1 \left(\hat{R}^{\theta}_{\text{out}} - \hat{R}_{\text{out}}\right) dt$$

The log weight log W equals the sum over the jumps along the path of the log of the model rate $\hat{R}^{\theta}$ divided by the reference rate $\hat{R}$, minus the integral over time of $\hat{R}^{\theta}_{\text{out}}$, the model's total rate of leaving the current state, minus $\hat{R}_{\text{out}}$, the reference total rate.

Now write the path weight when the length can change

With insertions, the total rate of leaving a state depends on its length. We must account for the waiting time in the path likelihood.

The path KL we derived has these two terms:

$$\log W = \sum_{\text{jumps}} \log \frac{\hat{R}^{\theta}}{\hat{R}} - \int_0^1 \left(\hat{R}^{\theta}_{\text{out}} - \hat{R}_{\text{out}}\right) dt$$

With the first term, we score the jumps that happened, and with the second term, the jumps that did not.

Now write the path weight when the length can change

With insertions, the total rate of leaving a state depends on its length. We must account for the waiting time in the path likelihood.

The path KL we derived has these two terms:

$$\log W = \sum_{\text{jumps}} \log \frac{\hat{R}^{\theta}}{\hat{R}} - \int_0^1 \left(\hat{R}^{\theta}_{\text{out}} - \hat{R}_{\text{out}}\right) dt$$

With the first term, we score the jumps that happened, and with the second term, the jumps that did not.

That second term is the $\ell(A, B) = A \log(A/B) - A + B$ staying contribution, which matters here.

Now write the path weight when the length can change

With insertions, the total rate of leaving a state depends on its length. We must account for the waiting time in the path likelihood.

The path KL we derived has these two terms:

$$\log W = \sum_{\text{jumps}} \log \frac{\hat{R}^{\theta}}{\hat{R}} - \int_0^1 \left(\hat{R}^{\theta}_{\text{out}} - \hat{R}_{\text{out}}\right) dt$$

With the first term, we score the jumps that happened, and with the second term, the jumps that did not.

A path includes how long nothing happens, so models with the same jump choices can still give it different probabilities.

A2D2: a path includes the waiting time

Toy path of duration 0.5 with one insertion, AG to ACG. At that jump the model rate is 2 and the reference rate is 1. Throughout this path, total exit rates are 3 and 2.

A2D2: a path includes the waiting time

Toy path of duration 0.5 with one insertion, AG to ACG. At that jump the model rate is 2 and the reference rate is 1. Throughout this path, total exit rates are 3 and 2.

$$\text{jump factor} = \frac{2}{1} = 2, \qquad \text{waiting factor} = \exp[-0.5(3-2)] = e^{-0.5}$$

A2D2: a path includes the waiting time

Toy path of duration 0.5 with one insertion, AG to ACG. At that jump the model rate is 2 and the reference rate is 1. Throughout this path, total exit rates are 3 and 2.

$$\text{jump factor} = \frac{2}{1} = 2, \qquad \text{waiting factor} = \exp[-0.5(3-2)] = e^{-0.5}$$

The observed jump favors the model by a factor of two. The waiting interval contributes a separate factor.

A2D2: a path includes the waiting time

Toy path of duration 0.5 with one insertion, AG to ACG. At that jump the model rate is 2 and the reference rate is 1. Throughout this path, total exit rates are 3 and 2.

$$\text{jump factor} = \frac{2}{1} = 2, \qquad \text{waiting factor} = \exp[-0.5(3-2)] = e^{-0.5}$$

The observed jump favors the model by a factor of two. The waiting interval contributes a separate factor.

$$W = 2e^{-0.5} \approx 1.213, \qquad \log W = \log 2 - 0.5 \approx 0.193$$

A2D2: a path includes the waiting time

Toy path of duration 0.5 with one insertion, AG to ACG. At that jump the model rate is 2 and the reference rate is 1. Throughout this path, total exit rates are 3 and 2.

$$\text{jump factor} = \frac{2}{1} = 2, \qquad \text{waiting factor} = \exp[-0.5(3-2)] = e^{-0.5}$$

The observed jump favors the model by a factor of two. The waiting interval contributes a separate factor.

$$W = 2e^{-0.5} \approx 1.213, \qquad \log W = \log 2 - 0.5 \approx 0.193$$

The complete likelihood ratio for this toy path is 1.213.

A2D2: a path includes the waiting time

Toy path of duration 0.5 with one insertion, AG to ACG. At that jump the model rate is 2 and the reference rate is 1. Throughout this path, total exit rates are 3 and 2.

$$\text{jump factor} = \frac{2}{1} = 2, \qquad \text{waiting factor} = \exp[-0.5(3-2)] = e^{-0.5}$$

The observed jump favors the model by a factor of two. The waiting interval contributes a separate factor.

$$W = 2e^{-0.5} \approx 1.213, \qquad \log W = \log 2 - 0.5 \approx 0.193$$

The complete likelihood ratio for this toy path is 1.213.

When the sequence can grow, how long it waits carries information alongside which edit occurs.

Here is the small molecule result in Table 1 of the paper

Table 1 ***Drug-like small molecule design results.*** *Metrics were computed for* 1000 *generated sequences, and the mean and standard deviations were computed across* 3 *seeds.*

Method	Validity (% total)	Uniqueness (% valid)	Quality (% total)	QED (↑)	Synthetic Accessibility (↓)	Diversity (↑)
GenMol w/o confidence	$98.467_{\pm 0.094}$	$99.763_{\pm 0.048}$	$73.667_{\pm 1.159}$	$0.763_{\pm 0.002}$	$3.024_{\pm 0.017}$	$0.859_{\pm 0.000}$
GenMol w/ confidence	$99.733_{\pm 0.047}$	$99.532_{\pm 0.125}$	$84.000_{\pm 0.566}$	$0.813_{\pm 0.004}$	$2.881_{\pm 0.020}$	$0.817_{\pm 0.000}$
Pre-trained (Any Length)	$95.800_{\pm 0.294}$	$99.582_{\pm 0.086}$	$44.167_{\pm 0.574}$	$0.641_{\pm 0.002}$	$3.401_{\pm 0.032}$	$0.908_{\pm 0.000}$
A2D2 w/o quality	$96.400_{\pm 0.535}$	$62.484_{\pm 0.452}$	$76.067_{\pm 0.896}$	$0.795_{\pm 0.005}$	$2.306_{\pm 0.024}$	$0.730_{\pm 0.003}$
A2D2 w/o insertion quality	$92.200_{\pm 0.082}$	$93.746_{\pm 0.335}$	$71.067_{\pm 0.411}$	$0.752_{\pm 0.003}$	$2.654_{\pm 0.014}$	$0.832_{\pm 0.001}$
A2D2 w/o unmasking quality	$98.600_{\pm 0.082}$	$85.801_{\pm 0.387}$	$82.367_{\pm 0.736}$	$0.818_{\pm 0.003}$	$2.753_{\pm 0.026}$	$0.789_{\pm 0.002}$
A2D2 w/ both quality	$94.533_{\pm 0.665}$	$93.695_{\pm 0.925}$	$71.300_{\pm 0.852}$	$0.762_{\pm 0.004}$	$2.870_{\pm 0.018}$	$0.843_{\pm 0.001}$

Tang,..,Chatterjee. *arXiv* (2026)

The pre-trained any-length model reaches 44.2% quality

Table 1 ***Drug-like small molecule design results.*** *Metrics were computed for* 1000 *generated sequences, and the mean and standard deviations were computed across* 3 *seeds.*

Method	Validity (% total)	Uniqueness (% valid)	Quality (% total)	QED (↑)	Synthetic Accessibility (↓)	Diversity (↑)
GenMol w/o confidence	$98.467_{\pm 0.094}$	$99.763_{\pm 0.048}$	$73.667_{\pm 1.159}$	$0.763_{\pm 0.002}$	$3.024_{\pm 0.017}$	$0.859_{\pm 0.000}$
GenMol w/ confidence	$99.733_{\pm 0.047}$	$99.532_{\pm 0.125}$	$84.000_{\pm 0.566}$	$0.813_{\pm 0.004}$	$2.881_{\pm 0.020}$	$0.817_{\pm 0.000}$
Pre-trained (Any Length)	$95.800_{\pm 0.294}$	$99.582_{\pm 0.086}$	$44.167_{\pm 0.574}$	$0.641_{\pm 0.002}$	$3.401_{\pm 0.032}$	$0.908_{\pm 0.000}$
A2D2 w/o quality	$96.400_{\pm 0.535}$	$62.484_{\pm 0.452}$	$76.067_{\pm 0.896}$	$0.795_{\pm 0.005}$	$2.306_{\pm 0.024}$	$0.730_{\pm 0.003}$
A2D2 w/o insertion quality	$92.200_{\pm 0.082}$	$93.746_{\pm 0.335}$	$71.067_{\pm 0.411}$	$0.752_{\pm 0.003}$	$2.654_{\pm 0.014}$	$0.832_{\pm 0.001}$
A2D2 w/o unmasking quality	$98.600_{\pm 0.082}$	$85.801_{\pm 0.387}$	$82.367_{\pm 0.736}$	$0.818_{\pm 0.003}$	$2.753_{\pm 0.026}$	$0.789_{\pm 0.002}$
A2D2 w/ both quality	$94.533_{\pm 0.665}$	$93.695_{\pm 0.925}$	$71.300_{\pm 0.852}$	$0.762_{\pm 0.004}$	$2.870_{\pm 0.018}$	$0.843_{\pm 0.001}$

Tang,..,Chatterjee. *arXiv* (2026)

After fine-tuning, that rises to 76.1%, with QED from 0.641 to 0.795

Table 1 ***Drug-like small molecule design results.*** *Metrics were computed for* 1000 *generated sequences, and the mean and standard deviations were computed across* 3 *seeds.*

Method	Validity (% total)	Uniqueness (% valid)	Quality (% total)	QED (↑)	Synthetic Accessibility (↓)	Diversity (↑)
GenMol w/o confidence	$98.467_{\pm 0.094}$	$99.763_{\pm 0.048}$	$73.667_{\pm 1.159}$	$0.763_{\pm 0.002}$	$3.024_{\pm 0.017}$	$0.859_{\pm 0.000}$
GenMol w/ confidence	$99.733_{\pm 0.047}$	$99.532_{\pm 0.125}$	$84.000_{\pm 0.566}$	$0.813_{\pm 0.004}$	$2.881_{\pm 0.020}$	$0.817_{\pm 0.000}$
Pre-trained (Any Length)	$95.800_{\pm 0.294}$	$99.582_{\pm 0.086}$	$44.167_{\pm 0.574}$	$0.641_{\pm 0.002}$	$3.401_{\pm 0.032}$	$0.908_{\pm 0.000}$
A2D2 w/o quality	$96.400_{\pm 0.535}$	$62.484_{\pm 0.452}$	$76.067_{\pm 0.896}$	$0.795_{\pm 0.005}$	$2.306_{\pm 0.024}$	$0.730_{\pm 0.003}$
A2D2 w/o insertion quality	$92.200_{\pm 0.082}$	$93.746_{\pm 0.335}$	$71.067_{\pm 0.411}$	$0.752_{\pm 0.003}$	$2.654_{\pm 0.014}$	$0.832_{\pm 0.001}$
A2D2 w/o unmasking quality	$98.600_{\pm 0.082}$	$85.801_{\pm 0.387}$	$82.367_{\pm 0.736}$	$0.818_{\pm 0.003}$	$2.753_{\pm 0.026}$	$0.789_{\pm 0.002}$
A2D2 w/ both quality	$94.533_{\pm 0.665}$	$93.695_{\pm 0.925}$	$71.300_{\pm 0.852}$	$0.762_{\pm 0.004}$	$2.870_{\pm 0.018}$	$0.843_{\pm 0.001}$

Tang,..,Chatterjee. *arXiv* (2026)

With the quality heads, we trade some of that back for validity and diversity

Table 1 ***Drug-like small molecule design results.*** *Metrics were computed for* 1000 *generated sequences, and the mean and standard deviations were computed across* 3 *seeds.*

Method	Validity (% total)	Uniqueness (% valid)	Quality (% total)	QED (↑)	Synthetic Accessibility (↓)	Diversity (↑)
GenMol w/o confidence	$98.467_{\pm 0.094}$	$99.763_{\pm 0.048}$	$73.667_{\pm 1.159}$	$0.763_{\pm 0.002}$	$3.024_{\pm 0.017}$	$0.859_{\pm 0.000}$
GenMol w/ confidence	$99.733_{\pm 0.047}$	$99.532_{\pm 0.125}$	$84.000_{\pm 0.566}$	$0.813_{\pm 0.004}$	$2.881_{\pm 0.020}$	$0.817_{\pm 0.000}$
Pre-trained (Any Length)	$95.800_{\pm 0.294}$	$99.582_{\pm 0.086}$	$44.167_{\pm 0.574}$	$0.641_{\pm 0.002}$	$3.401_{\pm 0.032}$	$0.908_{\pm 0.000}$
A2D2 w/o quality	$96.400_{\pm 0.535}$	$62.484_{\pm 0.452}$	$76.067_{\pm 0.896}$	$0.795_{\pm 0.005}$	$2.306_{\pm 0.024}$	$0.730_{\pm 0.003}$
A2D2 w/o insertion quality	$92.200_{\pm 0.082}$	$93.746_{\pm 0.335}$	$71.067_{\pm 0.411}$	$0.752_{\pm 0.003}$	$2.654_{\pm 0.014}$	$0.832_{\pm 0.001}$
A2D2 w/o unmasking quality	$98.600_{\pm 0.082}$	$85.801_{\pm 0.387}$	$82.367_{\pm 0.736}$	$0.818_{\pm 0.003}$	$2.753_{\pm 0.026}$	$0.789_{\pm 0.002}$
A2D2 w/ both quality	$94.533_{\pm 0.665}$	$93.695_{\pm 0.925}$	$71.300_{\pm 0.852}$	$0.762_{\pm 0.004}$	$2.870_{\pm 0.018}$	$0.843_{\pm 0.001}$

Tang,..,Chatterjee. *arXiv* (2026)

The length and the tokens come from the same rates

Table 1 ***Drug-like small molecule design results.*** *Metrics were computed for* 1000 *generated sequences, and the mean and standard deviations were computed across* 3 *seeds.*

Method	Validity (% total)	Uniqueness (% valid)	Quality (% total)	QED (↑)	Synthetic Accessibility (↓)	Diversity (↑)
GenMol w/o confidence	$98.467_{\pm 0.094}$	$99.763_{\pm 0.048}$	$73.667_{\pm 1.159}$	$0.763_{\pm 0.002}$	$3.024_{\pm 0.017}$	$0.859_{\pm 0.000}$
GenMol w/ confidence	$99.733_{\pm 0.047}$	$99.532_{\pm 0.125}$	$84.000_{\pm 0.566}$	$0.813_{\pm 0.004}$	$2.881_{\pm 0.020}$	$0.817_{\pm 0.000}$
Pre-trained (Any Length)	$95.800_{\pm 0.294}$	$99.582_{\pm 0.086}$	$44.167_{\pm 0.574}$	$0.641_{\pm 0.002}$	$3.401_{\pm 0.032}$	$0.908_{\pm 0.000}$
A2D2 w/o quality	$96.400_{\pm 0.535}$	$62.484_{\pm 0.452}$	$76.067_{\pm 0.896}$	$0.795_{\pm 0.005}$	$2.306_{\pm 0.024}$	$0.730_{\pm 0.003}$
A2D2 w/o insertion quality	$92.200_{\pm 0.082}$	$93.746_{\pm 0.335}$	$71.067_{\pm 0.411}$	$0.752_{\pm 0.003}$	$2.654_{\pm 0.014}$	$0.832_{\pm 0.001}$
A2D2 w/o unmasking quality	$98.600_{\pm 0.082}$	$85.801_{\pm 0.387}$	$82.367_{\pm 0.736}$	$0.818_{\pm 0.003}$	$2.753_{\pm 0.026}$	$0.789_{\pm 0.002}$
A2D2 w/ both quality	$94.533_{\pm 0.665}$	$93.695_{\pm 0.925}$	$71.300_{\pm 0.852}$	$0.762_{\pm 0.004}$	$2.870_{\pm 0.018}$	$0.843_{\pm 0.001}$

Tang,..,Chatterjee. *arXiv* (2026)

Now compare the models on the same five questions

	MDLM	D3PM	Uniform	Block	Simplex

Now compare the models on the same five questions

	MDLM	D3PM	Uniform	Block	Simplex
Forward process	$a \to m$ at rate $\beta(t)$	$Q_t = (1 - \beta_t)I + \beta_t \mathbf{1}\pi^\top$	redraw at rate $\beta(t)$	masking inside each block	Dirichlet beliefs

Now compare the models on the same five questions

	MDLM	D3PM	Uniform	Block	Simplex
Forward process	$a \to m$ at rate $\beta(t)$	$Q_t = (1 - \beta_t)I + \beta_t \mathbf{1}\pi^\top$	redraw at rate $\beta(t)$	masking inside each block	Dirichlet beliefs
Closed-form marginal	$\alpha_t \delta_{x0} + (1 - \alpha_t)\delta_m$	$x_0 Q_t$	$\alpha_t \delta_{x0} + (1 - \alpha_t)/K$	MDLM marginal per block	mean $\alpha_t e_{x0} + (1 - \alpha_t)\pi$

Now compare the models on the same five questions

	MDLM	D3PM	Uniform	Block	Simplex
Forward process	$a \to m$ at rate $\beta(t)$	$Q_t = (1 - \beta_t)I + \beta_t \mathbf{1}\pi^\top$	redraw at rate $\beta(t)$	masking inside each block	Dirichlet beliefs
Closed-form marginal	$\alpha_t \delta_{x0} + (1 - \alpha_t)\delta_m$	$x_0 Q_t$	$\alpha_t \delta_{x0} + (1 - \alpha_t)/K$	MDLM marginal per block	mean $\alpha_t e_{x0} + (1 - \alpha_t)\pi$
Where it ends	all masks	π	uniform tokens	all-mask block	beliefs around π

Now compare the models on the same five questions

	MDLM	D3PM	Uniform	Block	Simplex
Forward process	$a \to m$ at rate $\beta(t)$	$Q_t = (1 - \beta_t)I + \beta_t \mathbf{1}\pi^\top$	redraw at rate $\beta(t)$	masking inside each block	Dirichlet beliefs
Closed-form marginal	$\alpha_t \delta_{x0} + (1 - \alpha_t)\delta_m$	$x_0 \bar{Q}_t$	$\alpha_t \delta_{x0} + (1 - \alpha_t)/K$	MDLM marginal per block	mean $\alpha_t e_{x0} + (1 - \alpha_t)\pi$
Where it ends	all masks	π	uniform tokens	all-mask block	beliefs around π
Reverse via posterior	reveal at rate $w(t)d_\theta$	Bayes step averaged over $\tilde{p}_\theta$	ratios averaged over d_θ	MDLM rates given the prefix	kernel averaged over d_θ

Now compare the models on the same five questions

	MDLM	D3PM	Uniform	Block	Simplex
Forward process	$a \to m$ at rate $\beta(t)$	$Q_t = (1 - \beta_t)I + \beta_t \mathbf{1}\pi^\top$	redraw at rate $\beta(t)$	masking inside each block	Dirichlet beliefs
Closed-form marginal	$\alpha_t \delta_{x0} + (1 - \alpha_t)\delta_m$	$x_0 \bar{Q}_t$	$\alpha_t \delta_{x0} + (1 - \alpha_t)/K$	MDLM marginal per block	mean $\alpha_t e_{x0} + (1 - \alpha_t)\pi$
Where it ends	all masks	π	uniform tokens	all-mask block	beliefs around π
Reverse via posterior	reveal at rate $w(t)d_\theta$	Bayes step averaged over $\tilde{p}_\theta$	ratios averaged over d_θ	MDLM rates given the prefix	kernel averaged over d_θ
Loss	weighted cross-entropy	$L_{vb} + \lambda$ CE	full rate loss	sum of block losses	cross-entropy

Suppose you want to build a new discrete diffusion model

Suppose you want to build a new discrete diffusion model

A recipe for your own model

1. Choose a rate matrix that encodes which replacements make sense for your data.

Suppose you want to build a new discrete diffusion model

A recipe for your own model

1. Choose a rate matrix that encodes which replacements make sense for your data.
2. Compute its matrix exponential to get the forward marginal in closed form.

Suppose you want to build a new discrete diffusion model

A recipe for your own model

1. Choose a rate matrix that encodes which replacements make sense for your data.
2. Compute its matrix exponential to get the forward marginal in closed form.
3. Check where the process settles, so generation can start from that distribution.

Suppose you want to build a new discrete diffusion model

A recipe for your own model

1. Choose a rate matrix that encodes which replacements make sense for your data.
2. Compute its matrix exponential to get the forward marginal in closed form.
3. Check where the process settles, so generation can start from that distribution.
4. Write the reverse rates through the posterior over clean tokens, one position at a time.

Suppose you want to build a new discrete diffusion model

A recipe for your own model

1. Choose a rate matrix that encodes which replacements make sense for your data.
2. Compute its matrix exponential to get the forward marginal in closed form.
3. Check where the process settles, so generation can start from that distribution.
4. Write the reverse rates through the posterior over clean tokens, one position at a time.
5. Train a denoiser with the rate loss, which bounds the likelihood.

Suppose you want to build a new discrete diffusion model

A recipe for your own model

1. Choose a rate matrix that encodes which replacements make sense for your data.
2. Compute its matrix exponential to get the forward marginal in closed form.
3. Check where the process settles, so generation can start from that distribution.
4. Write the reverse rates through the posterior over clean tokens, one position at a time.
5. Train a denoiser with the rate loss, which bounds the likelihood.
6. Use Doob’s h-transform to condition generation on the properties you want.

Suppose you want to build a new discrete diffusion model

A recipe for your own model

1. Choose a rate matrix that encodes which replacements make sense for your data.
2. Compute its matrix exponential to get the forward marginal in closed form.
3. Check where the process settles, so generation can start from that distribution.
4. Write the reverse rates through the posterior over clean tokens, one position at a time.
5. Train a denoiser with the rate loss, which bounds the likelihood.
6. Use Doob's h-transform to condition generation on the properties you want.

If you can prove the five facts for your rate matrix, you have a discrete diffusion model.

Finally, let’s look back at this lecture

Finally, let’s look back at this lecture

We chose how to corrupt the data, and by time reversal we got the rates for generation.

Finally, let’s look back at this lecture

We chose how to corrupt the data, and by time reversal we got the rates for generation.

The reverse rates, the posterior average, and the loss all followed from that one choice.

Finally, let’s look back at this lecture

We chose how to corrupt the data, and by time reversal we got the rates for generation.

The reverse rates, the posterior average, and the loss all followed from that one choice.

In Lecture 5, we choose the path of distributions first, and construct rates whose master equation follows it.

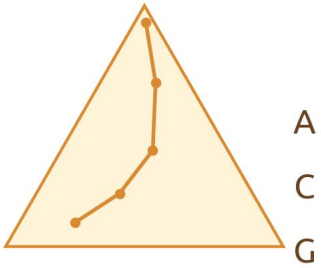
Finally, let’s look back at this lecture

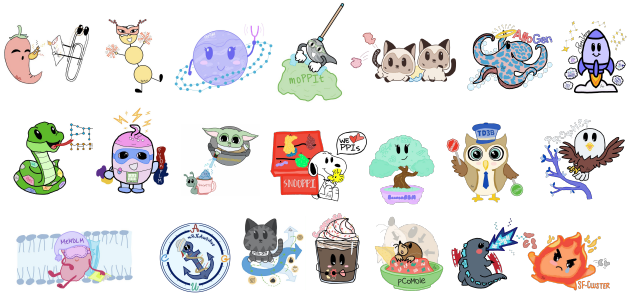
We chose how to corrupt the data, and by time reversal we got the rates for generation.

The reverse rates, the posterior average, and the loss all followed from that one choice.

In Lecture 5, we choose the path of distributions first, and construct rates whose master equation follows it.

The master equation, the posterior average, and the rate loss will all return.





And that leads us to
Discrete Flow Matching!

Penn Engineering
University of Pennsylvania

